

SUBSPACE-CONFIGURABLE NETWORKS

Dong Wang^{†,*}, Olga Saukh^{†,◇,*}, Xiaoxi He[‡], Lothar Thiele[§]

[†]Graz University of Technology, Austria, [◇]Complexity Science Hub Vienna, Austria

[‡]University of Macao, China, [§]ETH Zurich, Switzerland

{dong.wang, saukh}@tugraz.at, hexiaoxi@um.edu.mo, thiele@tik.ee.ethz.ch

ABSTRACT

While the deployment of deep learning models on edge devices is increasing, these models often lack robustness when faced with dynamic changes in sensed data. This can be attributed to sensor drift, or variations in the data compared to what was used during offline training due to factors such as specific sensor placement or naturally changing sensing conditions. Hence, achieving the desired robustness necessitates the utilization of either an invariant architecture or specialized training approaches, like data augmentation techniques. Alternatively, input transformations can be treated as a domain shift problem, and solved by post-deployment model adaptation. In this paper, we train a parameterized subspace of *configurable networks*, where an optimal network for a particular parameter setting is part of this subspace. The obtained subspace is low-dimensional and has a surprisingly simple structure even for complex, non-invertible transformations of the input, leading to an exceptionally high efficiency of subspace-configurable networks (SCNs) when limited storage and computing resources are at stake. Our source code is online.¹

1 INTRODUCTION

In real-world applications of deep learning, it is common for systems to encounter environments that differ from those considered during model training. There are many reasons for this difference between training and post-deployment such as sensor drift, device-to-device variation, and domain shift in the data compared to what was used during offline training due to factors like a different sensor placement or changing sensing conditions. To address the above challenge, there are two primary approaches: designing robust, invariant models and employing domain adaptation techniques. Both strategies aim to mitigate the performance degradation resulting from the discrepancies between the source and the target domains.

Invariant architectures focus on making the model robust, insensitive, or invariant to specific transformations of the input data. This can be achieved by various means, including training with data augmentation (Botev et al., 2022; Geiping et al., 2022), canonicalization of the input data (Jaderberg et al., 2015; Kaba et al., 2022), adversarial training (Engstrom et al., 2017), and designing network architectures that inherently incorporate the desired invariances (Marcus, 2018; Kauderer-Abrams, 2018). Domain adaptation, on the other hand, seeks to transfer the knowledge acquired from a source domain to a target domain, where the data distributions may differ. This approach leverages the learned representations or features from the source domain and fine-tunes or adapts them to better align with the target domain (Russo et al., 2017; Xu et al., 2018).

Unlike traditional invariant architectures, configurable networks explicitly define invariances by parameterizing desired input data transformations. We introduce subspace-configurable networks (SCNs), which are trained with weights residing in a subspace formed by a few base models. By receiving a parameter vector for an input transformation, SCNs select appropriate high-accuracy model weights from this subspace, effectively isolating the targeted invariances. We evaluate our SCN models by studying 2D translation, scaling, translation, and complex irreversible transformations like 3D rotation-and-projection. In the appendix, we also evaluate a wide range of real-world transformations covering both computer vision and audio signal processing domains, along with dedicated network architectures. To uncover configuration subspaces for a set of input transformations, SCNs leverage a hypernet-inspired architecture (Ha et al., 2016) to learn optimal inference models for each specific transformation in the set (Figure 1 left). To offer additional insights, we visualize the relation between the input transformation parameters and the configuration vector in the configuration subspace for a number of transformations (an example of the configuration subspace for 2D rotation is shown in Figure 1 middle). Interestingly, the configuration parameter vectors form well-structured geometric objects,

* Equal contribution

¹<https://github.com/osaukh/subspace-configurable-networks>

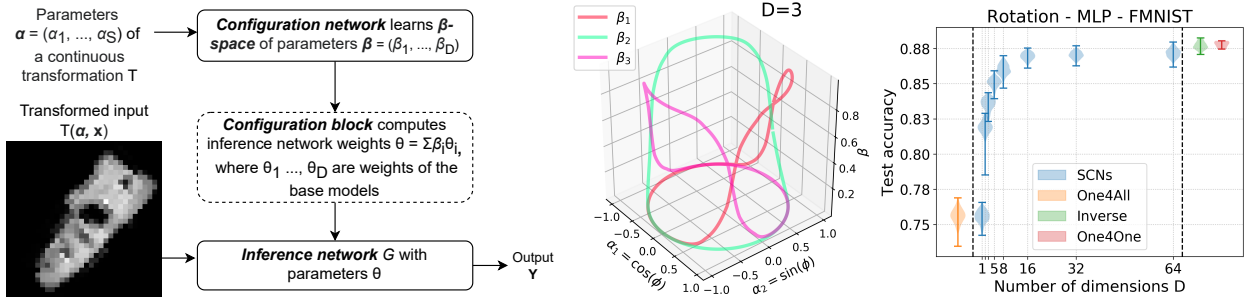


Figure 1: **Training subspace-configurable networks (SCNs)**, where an optimal network for a fixed transformation parameter vector is part of the subspace retained by few configuration parameters. **Left:** Given input transformation parameters α , e.g., a rotation angle for a 2D rotation, we train a *configuration network* which yields a D -dimensional configuration subspace (β -space) to construct an efficient inference network with weights $\theta = \sum \beta_i \cdot \theta_i$, where θ_i are the weights of the *base models*, and β is a *configuration vector*. **Middle:** Optimal model parameters in the configuration subspace as function of the rotation angle α given by $(\cos(\phi), \sin(\phi))$ for 2D rotation transformations applied to FMNIST (Xiao et al., 2017). Here SCN has three base models with parameters θ_i and three configuration vectors β_i to compose the weights of the 1-layer MLP inference model. **Right:** Test accuracy of SCNs with $D = 1..64$ dimensions compared to a single network trained with data augmentation (One4All), classifiers trained on canonicalized data achieved by applying inverse rotation transformation with the corresponding parameters (Inverse), and networks trained and tested on datasets where all images are rotated by a fixed degree (One4One).² Each violin shows the performance of a model on all degrees with a discretization step of 1° , expect for One4One where the models are independently trained and evaluated on $0, \pi/6, \pi/4, \pi/3, \pi/2$ rotated input.

highlighting the underlying structure of the optimal parameter subspaces. If the inference network capacity is fixed, usually due to severe resource constraints of edge devices, SCNs can quickly beat training with data augmentation (One4All) and match or outperform solutions trained for input transformation parameters optimized for each input transformation separately (One4One), see Figure 1 right. The contributions of this paper are summarized as follows:

- We design *subspace-configurable networks (SCNs)* to learn the configuration subspace and generate optimal networks for specific transformations. The approach presents a highly resource-efficient alternative to model adaptation through retraining and specifically targets resource-constrained devices. Our approach and theoretical insights are covered in Section 2 and Appendix A.
- SCNs are evaluated on ten common real-world transformations, using five backbones and five standard benchmark datasets from computer vision and audio signal processing domains. The results are detailed in Section 3 and Appendix C-E.
- SCNs take transformation parameters as input, yet these parameters can be estimated from the input data. We provide an algorithm to build a transformation-invariant model on top of SCNs in Section 2 and Appendix F.
- In practical IoT scenarios the parameter supply can be replaced with a correlated sensor modality. We implemented SCNs on two resource-constrained devices and show in Section 3.6 and Appendix G their outstanding performance and remarkable efficiency.

Section 4 concludes this paper with a discussion of limitations and outlining future research directions. Further related work is presented in Appendix H.

2 SUBSPACE-CONFIGURABLE NETWORKS

2.1 TRANSFORMATIONS AND THEIR PARAMETERIZATION

Let $\mathbb{X} \times \mathbb{Y} = \{(x, y)\}$ be a dataset comprising labelled examples $x \in \mathbb{X} \subset \mathbb{R}^N$ with class labels $y \in \mathbb{Y} \subset \mathbb{R}^M$. We apply a transformation $T : \mathbb{R}^S \times \mathbb{R}^N \rightarrow \mathbb{R}^N$ parameterized by the vector $\alpha = (\alpha_1, \dots, \alpha_S) \in \mathbb{A} \subseteq \mathbb{R}^S$ to each input example x . A transformed dataset is denoted as $T(\alpha, \mathbb{X}) \times \mathbb{Y} := \{(T(\alpha, x), y)\}$. For instance, let $\mathbb{X}$ be a collection

²One4One = one model for one parameter setting, i.e., a fixed rotation degree. One4All = one model for all parameter settings, i.e., a model trained with data augmentation for the considered range of parameter values.

of $P \times P$ images, then we have $x \in \mathbb{R}^{P^2}$ where each dimension corresponds to the pixel intensity of a single pixel. Transformation $T(\alpha, \mathbb{X}) : \mathbb{A} \times \mathbb{R}^{P^2} \rightarrow \mathbb{R}^{P^2}$ is modulated by pose parameters α , such as rotation, scaling, translation or cropping. We assume that data transformations $T(\alpha, \mathbb{X})$ preserve the label class of the input and represent a continuous function of $\alpha \in \mathbb{A}$, i.e., for any two transformation parameters α_1 and α_2 there exists a continuous curve in $\mathbb{A}$ that connects two transformation parameters. Note that by changing α we transform all relevant data distributions the same way, e.g., the data used for training and test. The set $\{T(\alpha, x) \mid \alpha \in \mathbb{A}\}$ of all possible transformations of input x is called an *orbit* of x . We consider an infinite orbit defined by a continuously changing α .

We consider an inference network to represent a function $g : \mathbb{X} \times \mathbb{R}^L \rightarrow \mathbb{Y}$ that maps data $x \in \mathbb{X}$ from the input space $\mathbb{X}$ to predictions $g(x, \theta) \in \mathbb{Y}$ in the output space $\mathbb{Y}$, where the mapping depends on the weights $\theta \in \mathbb{R}^L$ of the network. $E(\theta, \alpha)$ denotes the expected loss of the inference network and its function g over the considered training dataset $T(\alpha, \mathbb{X})$. Since the expected loss may differ for each dataset transformation parameterized by α , we write $E(\theta, \alpha)$ to make this dependency explicit. Optimal network parameters θ_α^* are those that minimize the loss $E(\theta, \alpha)$ for a given transformation vector α .

2.2 LEARNING CONFIGURABLE NETWORKS

The architecture of SCNs is sketched in Figure 1 (left). Excited by the hypernet (Ha et al., 2016) design, we train a configuration network with function $h(\cdot)$ and an inference network with function $g(\cdot)$ connected by a linear transformation of network parameters $\theta = f(\beta)$ computed in the configuration block:

$$\theta = f(\beta) = \sum_{i=1}^D \beta_i \cdot \theta_i, \quad (1)$$

where $\theta_i \in \mathbb{T} \subseteq \mathbb{R}^L$ for $i \in [1, D]$ denote the static weights (network parameters) of the base models that are the result of the training process. The configuration network with the function $h : \mathbb{R}^S \rightarrow \mathbb{R}^D$ yields a low-dimensional *configuration space* of vectors $\beta \in \mathbb{R}^D$, given transformation parameters $\alpha \in \mathbb{A}$. Along with learning the mapping h , we train the D *base models* with weights $\theta_i \in \mathbb{R}^L$ to construct the weights of inference networks $\theta = f(\beta)$. The SCN training process minimizes the expected loss $E(\theta, \alpha) = E(f(h(\alpha)), \alpha)$ to determine the configuration network with function h and the base model parameters θ_i . We use the standard categorical cross-entropy loss in all our experiments. During inference, a transformed input example $\hat{x} = T(\alpha, x)$ is classified by the inference network with weights θ , $\beta = h(\alpha)$ and $y = g(\hat{x}, f(h(\alpha)))$.

Note that degenerated solutions, where β is constant for all α are part of the solution space. In this case, SCN ends up representing a single model for all transformation parameters $\alpha \in \mathbb{A}$, which is essentially the One4All model, i.e., a model trained with data augmentation over all transformation parameters $\alpha \in \mathbb{A}$. To avoid degenerated cases, we enhance the cross-entropy loss function with a regularization term as a squared cosine similarity $\cos^2(\beta^{(1)}, \beta^{(2)})$ between the configuration vector $\beta^{(1)}$ for a randomly chosen $\alpha^{(1)}$ applied to transform the current batch, and a vector $\beta^{(2)}$ obtained from the configuration network for another randomly sampled $\alpha^{(2)} \in \mathbb{A}$. The applied regularization (with a weighting factor of 1.0) improves the performance of SCNs by reinforcing them to construct unique dedicated inference networks for different transformation parameters α .

For a continuous transformation $T(\alpha)$, in the next section and in Appendix A we provide theoretical results that help to understand the structure of the β -space using continuity arguments of the optimal solution space.

2.3 CONTINUITY OF THE LEARNED SUBSPACES

Figure 1 (middle) exemplifies a learned β -space for the 2D rotation transformation learned by SCN for $D = 3$ with a MLP inference network architecture trained on FMNIST. Transformation parameters $(\alpha_1, \alpha_2) = (\cos(\phi), \sin(\phi))$ with $\phi = 0..2\pi$ yield 3-dimensional β vectors $(\beta_1, \beta_2, \beta_3)$ with each β_i being in charge of a specific contiguous region of the α -space. Transitions between regions are covered by models that are a linear combination of optimal solutions for other α values. This structural property of the β -space is independent of the dataset and architecture we used to train SCNs, as shown in the next section. Moreover, we observe a continuous change of β as we change α .

Another observation we make from Figure 1 (right) is that SCNs match the high performance of the baselines already for small number of dimensions D of the linear subspace. In other words, the solution subspace spanned by only D base models learned by SCN pays no penalty for its very simple structure.

We provide theoretical results that help to understand the structure of the β -space using continuity arguments of the optimal solution space. Informally, the following theorem shows under certain conditions that for every continuous

curve connecting two transformation parameters in $\mathbb{A}$, there exists a corresponding continuous curve in the network parameter space $\mathbb{T}$. These two curves completely map onto each other where the network parameters are optimal for the corresponding data transformations. In particular, the curve in the network parameter space $\mathbb{T}$ is continuous.

To simplify the formulation of the theorems (see Appendix A.1 for the respective proofs), we suppose that the set of admissible parameters $\theta \in \mathbb{T}$ is a bounded subspace of $\mathbb{R}^L$ and all optimal parameter vectors (weights) θ_α^* are in the interior of $\mathbb{T}$.

Theorem 2.1 (Continuity). *Suppose that the loss function $E(\theta, \alpha)$ satisfies the Lipschitz condition*

$$|E(\theta, \alpha^{(2)}) - E(\theta, \alpha^{(1)})| \leq K_\alpha \|\alpha^{(2)} - \alpha^{(1)}\|_2 \quad (2)$$

for $\alpha^{(1)}, \alpha^{(2)} \in \mathbb{A}$, and $E(\theta, \alpha)$ is differentiable w.r.t. to θ and α . Then, for any continuous curve $\alpha(s) \in \mathbb{A}$ with $0 \leq s \leq \hat{s}$ in the parameter space of data transformations there exists a corresponding curve $\theta(t) \in \mathbb{T}$ with $0 \leq t \leq \hat{t}$ in the parameter space of network weights and a relation $(s, t) \in R$ such that

- *the domain and range of R are the intervals $s \in [0, \hat{s}]$ and $t \in [0, \hat{t}]$, respectively, and*
- *the relation R is monotone, i.e., if $(s_1, t_1), (s_2, t_s) \in R$ then $(s_1 \geq s_2) \Rightarrow (t_1 \geq t_2)$, and*
- *for every $(s, t) \in R$ the network parameter vector $\theta(t)$ minimizes the loss function $E(\theta, \alpha)$ for the data transformation parameter $\alpha(s)$.*

We are also interested in the relation between α and corresponding optimal vectors β that define optimal locations on the linear subspace of admissible network parameters as defined by (1). To simplify the formulation of the further theoretical result and proof, we suppose that $\beta \in \mathbb{B}$ where $\mathbb{B}$ is a bounded subspace of $\mathbb{R}^D$, and all basis vectors θ_j that define $f(\beta)$ in (1) have bounded elements. Under these assumptions, we can derive a corollary from Theorem 2.1.

Corollary 2.2. *Suppose that the loss function $E(\theta, \alpha)$ satisfies (2), and for any $\alpha \in \mathbb{A}$ all local minima of $E(f(\beta), \alpha)$ w.r.t. β are global. Then the following holds: For any continuous curve $\alpha(s) \in \mathbb{A}$ with $0 \leq s \leq \hat{s}$ in the parameter space of data transformations there exists a corresponding curve $\beta(t) \in \mathbb{B}$ with $0 \leq t \leq \hat{t}$ on the linear network parameter subspace according to (1) and a relation $(s, t) \in R$ such that*

- *the domain and range of R are the intervals $s \in [0, \hat{s}]$ and $t \in [0, \hat{t}]$, respectively, and*
- *the relation R is monotone, i.e., if $(s_1, t_1), (s_2, t_s) \in R$ then $(s_1 \geq s_2) \Rightarrow (t_1 \geq t_2)$, and*
- *for every $(s, t) \in R$ the network parameter vector $\beta(t)$ minimizes the loss function $E(f(\beta), \alpha)$ for the data transformation parameter $\alpha(s)$.*

The proof of the corollary is in Appendix A.1. The above corollary provides a theoretical argument for the continuous curves in Figure 1 (middle), i.e., the curves explicitly show the relation R . The existence of such a relation R is also apparent for all the dataset-architecture pairs used to empirically evaluate SCNs in Section 3.

Appendix A.2 contains a further result: Small changes to transform parameters α result in small changes of optimal configuration vectors β_α^* for suitable loss functions $E(f(\beta), \alpha)$. In other words, the relation R can be represented as a continuous function r with $t = r(s)$, i.e., the parameter vector $\beta(t)$ that minimizes the loss function can be determined as a function of the data transformation $\alpha(s)$.

2.4 SEARCH IN THE α -SPACE AND PRACTICAL VALUE OF SCNS

SCNs take transformation parameter α as input. However, one can also use a search algorithm to estimate α from the input data, aiming for low-entropy, confident classification results. This approach, inspired by previous techniques (Wortsman et al., 2020; Hendrycks & Gimpel, 2016), utilizes the basin hopping method Iwamatsu & Okabe (2004) targeting hard nonlinear optimization problems with a mix of global and local search phases. To enhance performance, SCN training includes a regularizer to optimize output entropy based on the correct α . Despite the computational demands of the α -search algorithm, it achieves high accuracy, allowing SCNs to serve as transformation-invariant networks. In practice, however, search in the α -space can often be avoided. If the transformation of interest is discrete and limited to a few cases, α -search can be reduced to running inference over a few candidate models. Most importantly, however, the parameter α can be inferred from a correlated sensor modality. We demonstrate this in two edge applications discussed in Section 3.6 and Appendix G.

Several peculiarities of SCNs' design make them well-suited for resource-constrained devices. (1) SCNs take advantage of memory hierarchies. Fast memory, such as SRAM, offers access times in the range of nanoseconds, but it is usually available in smaller capacities. In contrast, slow memory like Flash or EEPROM, provides larger storage

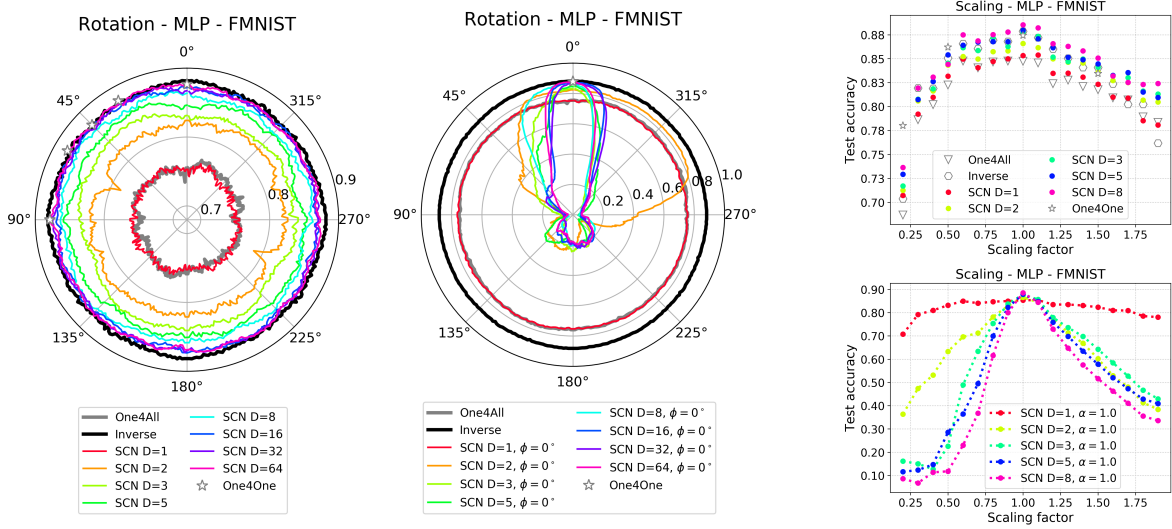


Figure 2: **SCN test accuracy for 2D rotation and scaling transformations.** **Left and middle:** 2D rotation parameterized by a rotation degree $\phi = 0..2\pi$ input to the configuration network as $\alpha = (\cos(\phi), \sin(\phi))$. For each α , SCN determines a configuration vector β used to build a dedicated model for every angle shown on the right. The middle polar plot shows the performance of a single model ($\phi = 0^\circ$) on all angles. The model works best for the input transformed with $T(\phi = 0^\circ)$. Inference network architecture is a 1-layer MLP with 32 hidden units trained on FMNIST. The models constructed by SCN outperform One4All approaching Inverse and One4One accuracy already for small D . **Right top:** Scaling transformation parameterized by the scaling factor $\alpha = 0.2..2.0$. **Right bottom:** SCN performance of a single model ($\alpha = 1.0$) on all inputs. The dedicated model gets increasingly specialized for the target input parameters with higher D . Inference network is a 5-layer MLP with 32 hidden units in each layer trained on FMNIST. Also see Appendix C and videos showing SCN inference models for each parameter setting.³

capacities, but with slower access times. Slow memory is ideal for storing less time-sensitive SCN reconfiguration data, such as the D base models holding θ_i and the parameters of the configuration network. At the same time, the inference network weights θ should better completely fit into RAM to support fast inference. (2) SCNs yield the most benefit if memory and thus the network capacity are limited. Given unlimited resources, One4All can match the performance of SCNs. Exceptions include corner cases where the transformation parameter α is used to break symmetries. (3) SCNs draw inspiration from the recent linear mode connectivity literature (Entezari et al., 2021; Ainsworth et al., 2022), and empirically show that a linear reconfiguration function $f(\beta)$ yields great performance of models for different transformation parameters. This decision allows implementing efficient memory access strategies for SCN reconfiguration, such as reuse of memory pages. Also computation of equation 1 can be efficiently realized with hardware-friendly linear vector operations, *i.e.*, using MLA and FMA instructions, vectorization, and pipelining.

3 EXPERIMENTAL RESULTS

We evaluate the performance of SCNs on 10 popular transformations (2D rotation, scaling, translation, brightness, contrast, saturation, sharpness, 3D rotation-and-projection, pitch shift and audio speed change) and five dataset-architecture pairs from computer vision and audio signal processing domains (MLPs on FMNIST (Xiao et al., 2017), ShallowCNNs (Neyshabur, 2020) on SVHN (Netzer et al., 2011), LeNet-5 (Lecun et al., 1998) on ModelNet10 (Wu et al., 2015), ResNet18 (He et al., 2015) on CIFAR10 (Krizhevsky et al., 2009), and M5 (Dai et al., 2016) on Google Speech Commands Dataset (Warden, 2018)). All considered transformations are continuous, and their parameterization is straightforward. For example, a rotation angle for a 2D rotation. The main paper exemplifies the obtained results to highlight SCN performance, while additional plots can be found in Appendix C-E. Training hyper-parameters, architectural choices, dataset description and samples of the transformed images are presented in Appendix B.

We compare SCNs to the following baselines. *One4All* represents a network trained with data augmentation obtained by transforming the input by randomly chosen parameters $\alpha \in \mathbb{A}$. *Inverse* classifier is trained on a canonicalized data representation achieved by first applying the inverse transformation to the transformed input. Note that 2D rotation is a

³<https://tinyurl.com/2nb8k644>

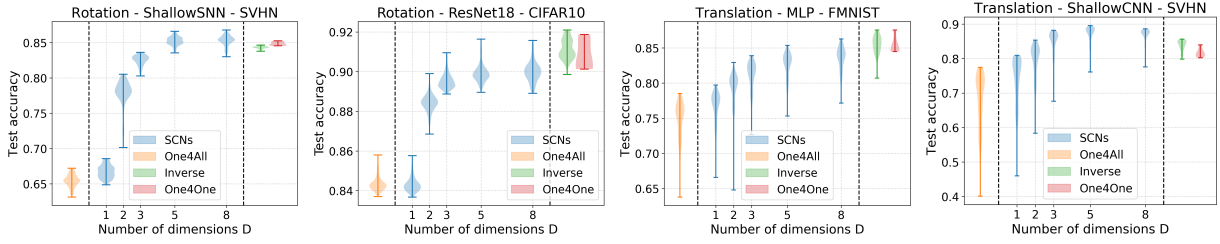


Figure 3: **SCNs achieve high test accuracy already for low D** , outperforming One4All and approaching (and in some cases outperforming) both Inverse and One4One baselines. **2 plots on the left:** 2D rotation on ShallowCNN–SVHN and ResNet18–CIFAR10. **2 plots on the right:** Scaling on MLP–FMNIST and ShallowCNN–SVHN. The plots are complementary to Figure 2 evaluating the performance of SCN on different transformations and dataset-architecture pairs. For translation, the violin for One4One comprises prediction accuracy of independently trained models for $(0,0)$ and $(\pm 8, \pm 8)$ shift parameters. A detailed evaluation of SCNs for translation is detailed in Appendix D.

fully invertible transformation in theory, yet introduces small distortions in practice due to rounding effects. Translation is fully invertible if the relevant part of the input stays within the input boundaries. Scaling brings significant distortion to the input, and inversion leads to a considerable loss of input quality. Finally, *One4One* represents a set of networks, each trained and tested on the dataset transformed with a fixed parameter vector α . Note that for a fixed architecture, dataset, and loss function, a well-trained One4One baseline achieves the best in-distribution generalization. In this sense, it upper bounds the performance which can be achieved by any domain adaptation method using the same data. When comparing model performance throughout this work, all baselines feature *the same* model architecture and have *the same* capacity as the SCN inference network. We use a 1-layer MLP with 64 hidden units as the configuration network architecture to learn the configuration subspace $\beta = h(\alpha)$. Our main evaluation metric is the test accuracy, but we also analyze the impact of SCN dimensionality D on its performance, and the structure of the β -space.

3.1 SCN TEST SET ACCURACY

Figure 2 and Figure 3 present different views on the SCN test accuracy as a function of the number of dimensions D when the concept is applied to different transformations, datasets, and architectures. Figure 2 left shows the performance of SCNs on $0 - 2\pi$ rotation angles. The test accuracy for $D = 1$ matches One4All but quickly approaches Inverse and One4One baselines for higher D . For the scaling transformation shown in Figure 2 top right, SCNs for $D > 1$ easily outperform One4All. They also outperform Inverse for $\alpha < 0.3$ and $\alpha > 1.2$ already for small D . Non-invertible transformations introduce significant distortion to the input data complicating feature re-use across inputs for different α . A large performance gap between One4One and Inverse for $\alpha = 0.2$ suggests that at small scales different features in the input become more relevant than in the original dataset. In some cases, SCNs achieve higher accuracy than One4One networks trained and tested only on the transformed data for some fixed value of α , since One4One does not make use of data augmentation but SCN implicitly does due to its structure given in Equation 1.

Figure 3 presents an aggregated view on the SCN test accuracy for 2D rotations on ShallowCNN–SVHN and ResNet18–CIFAR10, and also for translation on MLP–FMNIST and ShallowCNN–SVHN. Each violin comprises accuracies achieved by models tested on all parameter settings traversed with a very small discretization step (with a granularity of 1° and 1 pixel for 2D rotation and translation respectively). The only exception here is the One4One baseline, where a violin comprises the performance of five models independently trained and tested on the transformed inputs for a fixed parameter setting. The fixed parameters are chosen to cover $\mathbb{A}$ from the most beneficial (e.g., $\alpha = (0, 0)$ for translation) to the most suboptimal ($\alpha = (\pm 8, \pm 8)$ for translation) setting. This is why the violins for the translation transformation have a long tail of low accuracies. The performance of SCNs is consistent across dataset-architecture pairs, matching the best performing baselines already for a small number of dimensions D (also see Appendix C).

3.2 STRUCTURE OF THE CONFIGURATION SUBSPACE

The β -space learned by the configuration network h for different transformations, datasets, and inference network architectures is shown in Figure 4. For 2D rotation, the transformation parameters $\alpha = (\cos(\phi), \sin(\phi))$ are drawn from a circle and result in all β_i being continuous curves arranged around the cylinder in our α - β visualization. For all transformations, if $D = 1$, the SCN training yields $\beta_1 = 1$ due to the use of softmax in the last layer of the configuration network and a single base model. For $D \neq 1$, each β_i is high for a certain contiguous range of α s and low outside of this range. For small D , the regions of high β s are largely disjoint, yet overlap as D is scaled up. Interestingly, the

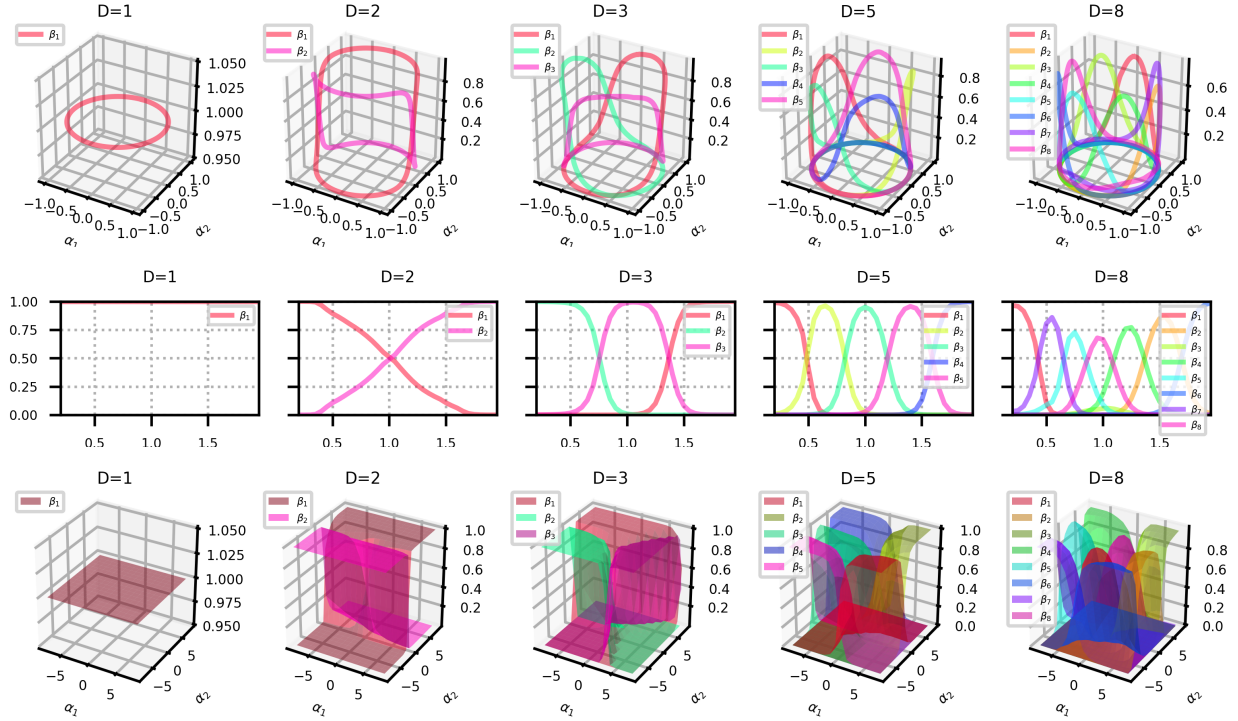


Figure 4: **A typical view of the β -space for 2D rotation, scaling and translation, $D = 1..8$.** The β -space is nicely shaped, with each β being responsible for a specific range of inputs with smooth transitions. **Top:** SCNs for 2D rotation on ResNet18–CIFAR10. Transformation parameters are a vector $\alpha = (\alpha_1, \alpha_2) = (\cos(\phi), \sin(\phi))$, with ϕ being a rotation angle. **Middle:** SCNs for scaling on ShallowCNN–SVHN, with a scaling factor α between 0.2 and 2.0. **Bottom:** SCNs for translation on MLP–FMNIST. A shift is specified by two parameters (α_x, α_y) varying in the range $(-8, 8)$ along x and y axes. A visualization for other dataset-architecture pairs is presented in Appendix C.2.

shape of the learned transformation is preserved across datasets and inference network architectures, although minor differences do exist, see Appendix C.2.

We claim that the subspace of optimized configurations for data transformations parameterized by α is *nicely structured*: (i) We achieve good accuracies even for a linear subspace of low degrees of freedom D . (ii) We observe a nice structure of optimal solutions in the space, as represented by the function $\beta = h(\alpha)$ and supported by our theoretical results. This finding is related to the recent literature on linear mode connectivity of independently trained solutions (Entezari et al., 2021), solutions that share a part of the training trajectory (Frankle et al., 2020), and those trained on data splits (Ainsworth et al., 2022; Jordan et al., 2022). SCNs establish linear mode connectivity between models trained for different transformation parameters, enhancing the existing literature.

Although the inference network architecture seems to have little impact on the shape of the learned β -space, there are interesting exceptions. How does configuration subspace look like if the inference network architecture is invariant to the applied transformation? We trained a SCN for translation with a translation-invariant CNN as inference network architecture. The learned configuration space, in this case, appears degenerated with only one translation-invariant model being learned, regardless of D , i.e., all but one β_i are zero for all transformation parameters α (see Appendix D).

3.3 SCN DIMENSIONALITY AND CAPACITY CONSTRAINTS

SCNs yield high performance already for low D , and we observe diminishing returns from adding dimensions for all tested transformations and dataset-architecture pairs, see Figure 3. SCN dimensionality D impacts the overhead of training SCN, including the weights of the configuration network to compute β s and the weights θ_i of the base models. It also affects the overhead of reconfiguring an inference model if the transformation parameters α change, e.g., to adapt an object detection model to a new camera position. Our empirical evaluation suggests that small D is sufficient to achieve high SCN performance.

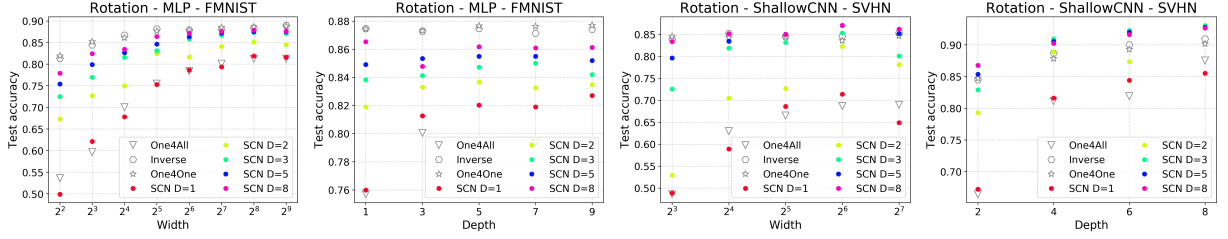


Figure 5: **Effect of network capacity on SCN test accuracy for 2D rotation.** We vary inference network width and depth to obtain the models of different capacity. **2 plots on the left:** Effect of width and depth for MLPs on FMNIST. In the width experiments, all networks are 1-layer MLPs. In the depth experiments, network width is fixed to 32 hidden units. **2 plots on the right:** Effect of width and depth for ShallowCNNs on SVHN. In the width experiments, the depth is fixed to two layers scaled together. In the depth experiments, the width of the hidden layers is fixed to 32 channels.

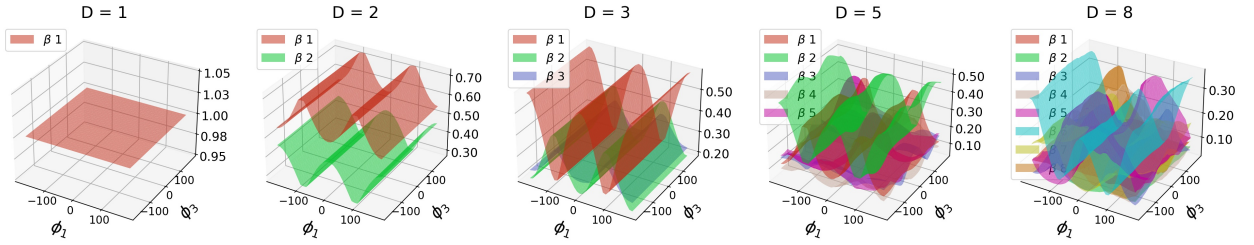


Figure 6: **A typical view of the SCN β -space for 3D rotation on LeNet5-ModelNet10.** Transformation parameters are a vector of ordered Euler angles (ϕ_1, ϕ_2, ϕ_3) , each taking values from $(-\pi, \pi)$. We show the learned β -space for $\phi_2 = -\pi$ with $D = 1..8$. Further views can be found in Appendix E. An interactive visualization is available⁴. The structure follows typical sine and cosine curves along multiple dimensions.

The optimal D depends on the inference network architecture and capacity. These trade-offs are explored when scaling inference network architectures along the width and depth dimensions in Figure 5. We present the capacity scaling results for the 2D rotation transformation for MLPs on FMNIST, and for ShallowCNNs on the SVHN dataset. Both architectures incorporate BatchNorm (Ioffe & Szegedy, 2015) layers in their deeper versions to facilitate training. Although increasing width proves to be more effective for all baselines for MLPs on FMNIST, higher depth leads to better test accuracy for ShallowCNNs on SVHN. Even for small values of D , SCNs consistently deliver performance improvements that are on par with the high accuracy achieved by models trained with specific parameter settings. We note that as capacity constraints get increasingly relaxed, One4All models can quickly improve performance approaching One4One and Inverse.

3.4 3D ROTATION-AND-PROJECTION TRANSFORMATION

We evaluate SCNs on 3D rotations that present complex transformations with multiple suboptimal views of the object that hurt classification accuracy. We sample a point cloud from a 3D object representation part of the MobileNet10 dataset, rotate it using a vector of Euler angles (ϕ_1, ϕ_2, ϕ_3) , and then project the point cloud to a 2D plane. The projection is then used as input to a classifier. We use LeNet5 as a backbone for the inference network to train SCNs. Figure 6 presents the view of the β -space as a function of two rotation angles ϕ_1 and ϕ_3 , while ϕ_2 is fixed at $-\pi$. The configuration space nicely reflects the characteristics of the input α , provided as $\sin(\cdot)$ and $\cos(\cdot)$ of the input angles.

Learned β s are insensitive to changes of ϕ_3 . Here ϕ_3 corresponds to object rotations in the plane that does not change the object’s visibility and thus leads to stable classification predictions, similarly to the 2D

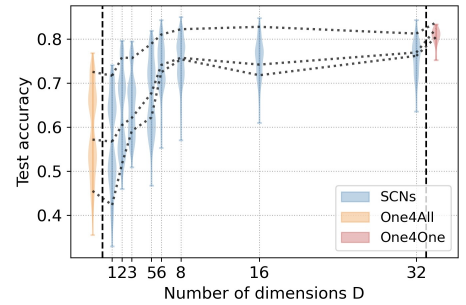


Figure 7: **Impact of D on the SCN’s test accuracy for 3D rotation.** Each line is associated with a specific test angle and connects accuracies tested on the same rotation. Some rotations of a 3D object lead to a suboptimal view of the object and may significantly hurt classification accuracy. With increasing D , SCNs outperform One4All and approach the One4One baseline.

⁴<https://subspace-configurable-networks.pages.dev/>

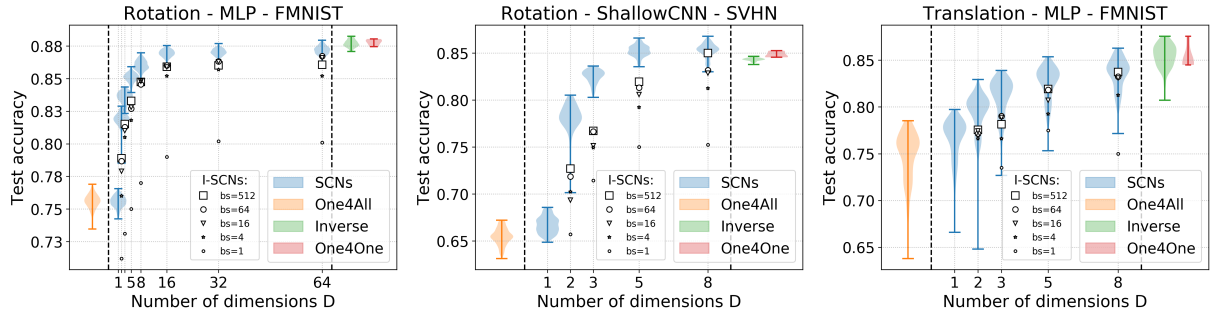


Figure 8: **Performance of the search algorithm in the α -space.** We enhance SCN evaluation plots in Figure 1 right and Figure 3 left and middle right with the performance of the presented search algorithm in the α -space. For higher batch sizes (≥ 4) the search algorithm performs close to the respective SCNs with known α .

rotation transformation of a flat image. The effect is best visible for low D and can be verified using the interactive visualization we provide and by inspecting further graphics in Appendix E.

Figure 7 compares SCN to One4All and One4One baselines. The Inverse is not feasible due to a projection of the point cloud on the 2D plane. Each violin comprises the model test accuracy evaluated on 30 randomly chosen angles. By comparing the accuracy for the same rotation angle (dotted lines in the plot), we observe a positive correlation between D and SCN test accuracy. The result is similar to the SCN performance on 2D transformations.

3.5 SEARCH IN THE α -SPACE AND I-SCNs

In Figure 8, we enhance three plots from Figure 1 and Figure 3 to show the performance of the search in the α -space. Note that the proposed input-based search algorithm allows constructing invariant SCNs, which we refer to as I-SCNs. We compare to the test accuracy achieved by the respective SCNs with known and correct input α to I-SCNs. The search algorithm operates on batches (bs = batch size). Batch size ≥ 4 allows for an accurate estimation of α from the input data and yields high I-SCN performance.

Network architectures can be designed to be invariant to transformations. For example, to achieve rotation invariance in 2D and 3D, an element-wise max-pooling layer can be utilized (Laptev et al., 2016; Su et al., 2015; Savva et al., 2016). TI-Pooling (called Transformation-Invariant Pooling) model (Laptev et al., 2016) employs parallel Siamese network layers with shared weights. We compare SCN and TI-Pooling models trained on 2D rotations with ϕ in the range $(0, \pi)$ on the FMNIST dataset. For SCNs, the inference network architecture is a 3-layer MLP with 64 hidden units in each layer. Tab. 1 shows the average classification accuracy and the number of parameters. SCN with $D=16$ dimensions demonstrates greater parameter efficiency compared to TI-Pooling, while also achieving higher accuracy than the baseline model.

Table 1: **Comparison of SCN to rotation-invariant TI-Pooling network.** With $D=16$ SCN is more parameter-efficient and yields higher accuracy than the baseline.

Model	Test accuracy [%]	#parameters
TI-Pooling	88.03	13'308'170
SCN($D=4$) [ours]	87.37	374'582
SCN($D=8$) [ours]	88.04	674'146
SCN($D=16$) [ours]	88.42	1'273'274

3.6 SCNs ON LOW-RESOURCE DEVICES

Embedded devices are frequently used in IoT applications on the edge. These devices feature severe resource constraints while running machine learning models, which should be robust to input transformations. We evaluate SCNs on two IoT applications: fruit classification using RGB sensor shown below, and traffic sign classification from camera images in Appendix G. We use Arduino Nano 33 BLE Sense (Arduino, 2023) to evaluate SCN performance on the fruit classification task. On-device APDS-9960 sensor is used to gather RGB and light intensity data, targeting bananas, apples, oranges, lemons, and kiwis under fluctuating natural light conditions. Figure 9 shows the performance of SCN ($D = 3$) and SCN ($D = 5$) on classifying fruits using RGB data parameterized by the light intensity transformation. The One4All model used in this experiment has the same architecture as the SCN base model and features a LeNet5-like layout. SCN reconfiguration overhead includes computation of parameters ("Hypernet Inference") and re-computation of inference model parameters ("Configuration"). The base model weights are stored in the on-board flash. Notice, that

the wider and deeper One4All variants lead to a higher resource consumption than SCNs, yet they perform considerably worse on the fruit classification task due to color ambiguity under varying light conditions.

4 CONCLUSION, LIMITATIONS, AND FUTURE WORK

This paper addresses the problem of model reconfiguration and robustness to input transformations under severe resource constraints. We design subspace-configurable networks (SCNs) that learn low-dimensional configuration subspaces and draw optimal model weights from this structure. We achieve surprisingly high accuracies even for a low number of configuration dimensions and observe a simple and intuitive structure of the subspace of optimal solutions for all investigated input transformations. Our findings open up practical applications of SCNs summarized below.

Post-deployment adaptation. SCNs can be used for the post-deployment model adaptation on resource-constrained devices as an alternative to costly backpropagation. SCN-configured inference networks are compact and can easily be deployed on devices with limited memory and processing power, *e.g.*, in robotics applications, edge computing, or classical embedded systems. In Section 3.6 and Appendix G we evaluate SCNs on two off-the-shelf MCUs (Tensilica Xtensa 32-bit LX7 dual-core and nRF52840) within the context of two example IoT applications: fruit classification using RGB sensor, and traffic sign classification from camera images. SCNs achieve a remarkable $\times 2.4$ RAM savings and $\times 7.6$ faster inference time compared to the One4All baseline reporting the same or higher test set accuracy.

SCNs as invariant architectures. SCNs can be used to build invariant network architectures on top of them by replacing the supply of α parameter vector with a search algorithm operating in the α -space. We leverage the fact that the correct input parameters α should produce a confident low-entropy classification result (Wortsman et al., 2020; Hendrycks & Gimpel, 2016). Note that SCNs without the α -search algorithm are of interest in their own right, since various sensor modalities can serve as input parameter α , *e.g.*, using IMU (Inertial Measurement Unit) sensor measurements to determine the current 2D rotation parameter α .

Limitations and future work. One of the major difficulties of this work is to effectively train SCNs for high D . The effects of the learning rate and training schedule are significant. With carefully chosen hyperparameters, we were able to train SCNs with $D = 32$ for 3D rotation using LeNet5 without any degenerated dimension. Although the current work mainly explores continuous transformations, the extension of SCNs to discrete transformations is imaginable, yet requires rethinking of the theoretical arguments to provide a better understanding of the SCN design choices, obtained performance and limitations. Also, the generalization ability of SCN on out-of-distribution data presents the direction of our future research. Furthermore, SCNs can achieve enhanced optimization when combined with parameter-efficient model update methods, which we plan to explore in our future research. Finally, SCNs require the knowledge of a correct α and may present an additional vector of input manipulation and adversarial attacks. This direction requires future research and should be carefully considered before SCNs can be safely deployed.

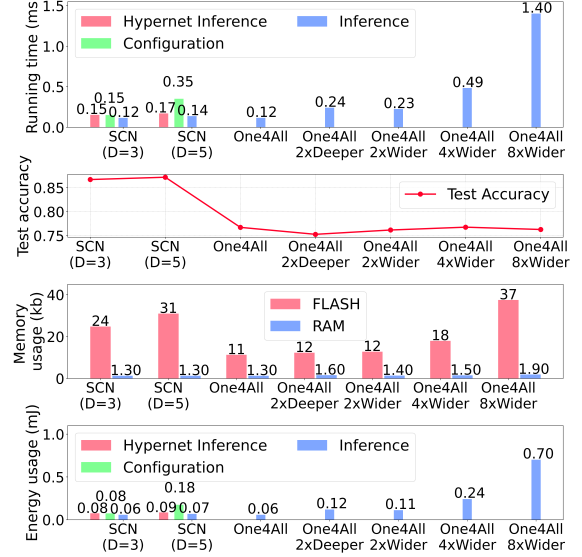


Figure 9: **SCN performance on the fruit classification task on Arduino Nano 33 BLE Sense.** From top to bottom the plots show: (1) Inference time in milliseconds, showcasing the efficiency of SCN, where the deeper and wider One4All variants lead to increased inference times. For SCNs, we also measure the execution latencies of the configuration network used to obtain vector β ("Hypernet Inference"), and the computation time for generating θ from base models θ_i ("Configuration"). These latencies are only incurred when the deployment environment changes. (2) Test accuracy across various architectures, highlighting SCN's highly competitive performance. (3) RAM and flash memory usage in kB, indicating the increased resource consumption as the One4All model expands. (4) Energy consumption in mJ.

ACKNOWLEDGEMENTS

We thank Mitchell Wortsman and Rahim Entezari for their insightful comments on the early draft of the manuscript. This work was partly funded by the Austrian Research Promotion Agency (FFG) and Pro2Future (STRATP II 4.1.4 E-MINDS strategic project). The results presented in this paper were computed using computational resources of ETH Zurich and the HLR resources of the Zentralen Informatikdienstes of Graz University of Technology.

REFERENCES

- Samuel K Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git Re-Basin: Merging models modulo permutation symmetries. *arXiv preprint*, 2022. URL <https://arxiv.org/abs/2209.04836>.
- Zeyuan Allen-Zhu, Yanzhi Li, and Zhao Song. A convergence theory for deep learning via over-parameterization. In *International Conference on Machine Learning*, pp. 242–252. PMLR, 2019.
- Arduino. Nano 33 ble sense, 2023. <https://docs.arduino.cc/hardware/nano-33-ble-sense> [Accessed: (2023-10-12)].
- Valerio Biscione and Jeffrey S. Bowers. Convolutional neural networks are not invariant to translation, but they can learn to be. *J. Mach. Learn. Res.*, 22(1), jul 2022. ISSN 1532-4435.
- Ryan Blything, Valerio Biscione, Ivan I. Vankov, Casimir J. H. Ludwig, and Jeffrey S. Bowers. The human visual system and cnns can both support robust online translation tolerance following extreme displacements. *arXiv preprint*, 2020. URL <https://arxiv.org/abs/2009.12855>.
- Aleksander Botev, Matthias Bauer, and Soham De. Regularising for invariance to data augmentation improves supervised learning. *arXiv preprint*, 2022. URL <https://arxiv.org/abs/2203.03304>.
- Lucas Caccia, Edoardo Ponti, Zhan Su, Matheus Pereira, Nicolas Le Roux, and Alessandro Sordoni. Multi-head adapter routing for cross-task generalization. *arXiv preprint arXiv:2211.03831*, 2022.
- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. *arXiv preprint*, 2020. URL <https://arxiv.org/abs/2002.05709>.
- Taco S. Cohen and Max Welling. Group equivariant convolutional networks. *arXiv preprint*, 2016. URL <https://arxiv.org/abs/1602.07576>.
- Wei Dai, Chia Dai, Shuhui Qu, Juncheng Li, and Samarjit Das. Very deep convolutional neural networks for raw waveforms. *CoRR*, 2016. URL <http://arxiv.org/abs/1610.00087>.
- Tri Dao, Albert Gu, Matthew Eichhorn, Atri Rudra, and Christopher Ré. Learning fast algorithms for linear transforms using butterfly factorizations. *arXiv preprint*, 2019. URL <https://arxiv.org/abs/1903.05895>.
- Logan Engstrom, Brandon Tran, Dimitris Tsipras, Ludwig Schmidt, and Aleksander Madry. Exploring the landscape of spatial robustness. *arXiv preprint*, 2017. URL <https://arxiv.org/abs/1712.02779>.
- Rahim Entezari, Hanie Sedghi, Olga Saukh, and Behnam Neyshabur. The role of permutation invariance in linear mode connectivity of neural networks. *arXiv preprint*, 2021. URL <https://arxiv.org/abs/2110.06296>.
- Espressif. Esp32-s3-eye get started, 2023. https://github.com/espressif/esp-who/blob/master/docs/en/get-started/ESP32-S3-EYE_Getting_Started_Guide.md [Accessed: (2023-10-12)].
- Carlos Esteves, Christine Allen-Blanchette, Xiaowei Zhou, and Kostas Daniilidis. Polar transformer networks. *arXiv preprint*, 2017. URL <https://arxiv.org/abs/1709.01889>.
- Stanislav Fort, Huiyi Hu, and Balaji Lakshminarayanan. Deep ensembles: A loss landscape perspective. *arXiv preprint*, 2019. URL <https://arxiv.org/abs/1912.02757>.
- Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *International Conference on Machine Learning*, pp. 3259–3269. PMLR, 2020.
- Kanchana Vaishnavi Gandikota, Jonas Geiping, Zorah Löhner, Adam Czaplinski, and Michael Möller. Training or architecture? how to incorporate invariance in neural networks. *CoRR*, abs/2106.10044, 2021.

- Ze-Feng Gao, Peiyu Liu, Wayne Xin Zhao, Zhong-Yi Lu, and Ji-Rong Wen. Parameter-efficient mixture-of-experts architecture for pre-trained language models. *arXiv preprint arXiv:2203.01104*, 2022.
- Mario Geiger, Stefano Spigler, Stéphane d’Ascoli, Levent Sagun, Marco Baity-Jesi, Giulio Biroli, and Matthieu Wyart. Jamming transition as a paradigm to understand the loss landscape of deep neural networks. *Physical Review E*, 100(1):012115, 2019.
- Jonas Geiping, Micah Goldblum, Gowthami Somepalli, Ravid Shwartz-Ziv, Tom Goldstein, and Andrew Gordon Wilson. How much data are augmentations worth? An investigation into scaling laws, invariance, and implicit regularization. *arXiv preprint*, 2022. URL <https://arxiv.org/abs/2210.06441>.
- Yunchao Gong, Liwei Wang, Ruiqi Guo, and Svetlana Lazebnik. Multi-scale orderless pooling of deep convolutional activation features. *arXiv preprint*, 2014. URL <https://arxiv.org/abs/1403.1840>.
- David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks. *arXiv preprint*, 2016. URL <https://arxiv.org/abs/1609.09106>.
- Fengxiang He and Dacheng Tao. Recent advances in deep learning theory. *arXiv preprint*, 2020. URL <https://arxiv.org/abs/2012.10931>.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. URL <http://arxiv.org/abs/1512.03385>.
- Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. *arXiv preprint*, 2016. URL <https://arxiv.org/abs/1610.02136>.
- Sebastian Houben, Johannes Stallkamp, Jan Salmen, Marc Schlipsing, and Christian Igel. Detection of traffic signs in real-world images: The German Traffic Sign Detection Benchmark. In *International Joint Conference on Neural Networks*, number 1288, 2013.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pp. 448–456. PMLR, 2015.
- Masao Iwamatsu and Yutaka Okabe. Basin hopping with occasional jumping. *Chemical Physics Letters*, 399(4-6): 396–400, 2004. doi: 10.1016/j.cplett.2004.10.032.
- Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- Max Jaderberg, Karen Simonyan, Andrew Zisserman, and Koray Kavukcuoglu. Spatial transformer networks. *arXiv preprint*, 2015. URL <https://arxiv.org/abs/1506.02025>.
- Xu Jiaolong, Xiao Liang, and Antonio M. López. Self-supervised domain adaptation for computer vision tasks. *IEEE Access*, 7:156694–156706, 2019.
- Keller Jordan, Hanie Sedghi, Olga Saukh, Rahim Entezari, and Behnam Neyshabur. REPAIR: Renormalizing permuted activations for interpolation repair. *arXiv preprint*, 2022. URL <https://arxiv.org/abs/2211.08403>.
- Jeevesh Juneja, Rachit Bansal, Kyunghyun Cho, João Sedoc, and Naomi Saphra. Linear connectivity reveals generalization strategies. *arXiv preprint*, 2022. URL <https://arxiv.org/abs/2205.12411>.
- Sékou-Oumar Kaba, Arnab Kumar Mondal, Yan Zhang, Yoshua Bengio, and Siamak Ravanbakhsh. Equivariance with learned canonicalization functions. In *NeurIPS 2022 Workshop on Symmetry and Geometry in Neural Representations*, 2022. URL <https://openreview.net/forum?id=pVD1k8ge25a>.
- Eric Kauderer-Abrams. Quantifying translation-invariance in convolutional neural networks. *arXiv preprint*, 2018. URL <https://arxiv.org/abs/1801.01450>.
- Kenji Kawaguchi and Qingyun Sun. A recipe for global convergence guarantee in deep neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pp. 8074–8082, 2021.
- Kenji Kawaguchi, Jiaoyang Huang, and Leslie Pack Kaelbling. Effect of depth and width on local minima in deep learning. *Neural computation*, 31(7):1462–1498, 2019.

- Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. *arXiv preprint*, 2020. URL <https://arxiv.org/abs/2004.11362>.
- Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. CIFAR-100 and CIFAR-10 (Canadian Institute for Advanced Research), 2009. URL <http://www.cs.toronto.edu/~kriz/cifar.html>. MIT License.
- Dmitry Laptev, Nikolay Savinov, Joachim M. Buhmann, and Marc Pollefeys. TI-POOLING: transformation-invariant pooling for feature learning in convolutional neural networks. *CoRR*, abs/1604.06318, 2016. URL <http://arxiv.org/abs/1604.06318>.
- Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Luca Della Libera, Vladimir Golkov, Yue Zhu, Arman Mielke, and Daniel Cremers. Deep learning for 2D and 3D rotatable data: An overview of methods. *CoRR*, 2019. URL <http://arxiv.org/abs/1910.14594>.
- Bo Liu. Spurious local minima are common for deep neural networks with piecewise linear activations. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.
- Mohammad Reza Loghmani, Luca Robbiano, Mirco Planamente, Kiru Park, Barbara Caputo, and Markus Vincze. Unsupervised domain adaptation through inter-modal rotation for RGB-D object recognition. *IEEE Robotics and Automation Letters*, abs/2004.10016, 2020. URL <https://arxiv.org/abs/2004.10016>.
- R. Manthalkar, P. K. Biswas, and B. N. Chatterji. Rotation and scale invariant texture features using discrete wavelet packet transform. *Pattern Recogn. Lett.*, 24(14):2455–2462, oct 2003. ISSN 0167-8655. doi: 10.1016/S0167-8655(03)00090-4.
- Diego Marcos, Michele Volpi, Nikos Komodakis, and Devis Tuia. Rotation equivariant vector field networks. *CoRR*, 2016. URL <http://arxiv.org/abs/1612.09346>.
- Gary Marcus. Deep learning: A critical appraisal. *arXiv preprint*, 2018. URL <https://arxiv.org/abs/1801.00631>.
- Vaishnavh Nagarajan and Zico Kolter. Deterministic PAC-Bayesian generalization bounds for deep networks via generalizing noise-resilience. In *Proceedings of the International Conference on Learning Representations*, 2019.
- Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011. URL <http://ufldl.stanford.edu/housenumbers>. License: CC0: Public Domain.
- Behnam Neyshabur. Towards learning convolutions from scratch. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2007.13657>.
- Quynh Nguyen, Mahesh Chandra Mukkamala, and Matthias Hein. On the loss landscape of a class of deep neural networks with no bad local valleys. *arXiv preprint*, 2018. URL <https://arxiv.org/abs/1809.10749>.
- Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. *arXiv preprint*, 2016. URL <https://arxiv.org/abs/1612.00593>.
- Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. *arXiv preprint*, 2017. URL <https://arxiv.org/abs/1706.02413>.
- Paolo Russo, Fabio Maria Carlucci, Tatiana Tommasi, and Barbara Caputo. From source to target and back: symmetric bi-directional adaptive GAN. *CoRR*, 2017. URL <http://arxiv.org/abs/1705.08824>.
- M. Savva, F. Yu, Hao Su, M. Aono, B. Chen, D. Cohen-Or, W. Deng, Hang Su, S. Bai, X. Bai, N. Fish, J. Han, E. Kalogerakis, E. G. Learned-Miller, Y. Li, M. Liao, S. Maji, A. Tatsuma, Y. Wang, N. Zhang, and Z. Zhou. Large-Scale 3D Shape Retrieval from ShapeNet Core55. In A. Ferreira, A. Giachetti, and D. Giorgi (eds.), *Eurographics Workshop on 3D Object Retrieval*. The Eurographics Association, 2016. ISBN 978-3-03868-004-8. doi: 10.2312/3dor.20161092.
- Arne Schneuing, Yuanqi Du, Charles Harris, Arian Jamasb, Ilia Igashov, Weitao Du, Tom Blundell, Pietro Lió, Carla Gomes, Max Welling, Michael Bronstein, and Bruno Correia. Structure-based drug design with equivariant diffusion models, 2022. URL <https://arxiv.org/abs/2210.13695>.

- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Patrice Simard, Bernard Victorri, Yann LeCun, and John Denker. Tangent prop - a formalism for specifying selected invariances in an adaptive network. In J. Moody, S. Hanson, and R.P. Lippmann (eds.), *NeurIPS*, volume 4. Morgan-Kaufmann, 1991.
- Berfin Şimşek, François Ged, Arthur Jacot, Francesco Spadaro, Clément Hongler, Wulfram Gerstner, and Johanni Brea. Geometry of the loss landscape in overparameterized neural networks: Symmetries and invariances. *arXiv preprint arXiv:2105.12221*, 2021.
- Hang Su, Subhransu Maji, Evangelos Kalogerakis, and Erik G. Learned-Miller. Multi-view convolutional neural networks for 3d shape recognition. In *Proc. ICCV*, 2015.
- Kai Sheng Tai, Peter Bailis, and Gregory Valiant. Equivariant transformer networks. *arXiv preprint*, 2019. URL <https://arxiv.org/abs/1901.11399>.
- Nathaniel Thomas, Tess Smidt, Steven Kearnes, Lusann Yang, Li Li, Kai Kohlhoff, and Patrick Riley. Tensor field networks: Rotation- and translation-equivariant neural networks for 3D point clouds. *arXiv preprint*, 2018. URL <https://arxiv.org/abs/1802.08219>.
- Bastiaan S. Veeling, Jasper Linmans, Jim Winkens, Taco Cohen, and Max Welling. Rotation equivariant CNNs for digital pathology. *arXiv preprint*, 2018. URL <https://arxiv.org/abs/1806.03962>.
- D. Vucetic, M. Tayarani, M. Ziaeeafard, J. Clark, B. Meyer, and W. Gross. Efficient fine-tuning of BERT models on the edge. In *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, may 2022. URL <https://doi.org/10.1109%2Fiscas48785.2022.9937567>.
- Rui Wang, Robin Walters, and Rose Yu. Approximately equivariant networks for imperfectly symmetric dynamics. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162, pp. 23078–23091. PMLR, 2022. URL <https://proceedings.mlr.press/v162/wang22aa.html>.
- Pete Warden. Speech commands: A dataset for limited-vocabulary speech recognition. *CoRR*, 2018. URL <http://arxiv.org/abs/1804.03209>.
- Maurice Weiler and Gabriele Cesa. General $e(2)$ -equivariant steerable CNNs. *arXiv preprint*, 2019. URL <https://arxiv.org/abs/1911.08251>.
- Maurice Weiler, Mario Geiger, Max Welling, Wouter Boomsma, and Taco Cohen. 3D Steerable CNNs: Learning rotationally equivariant features in volumetric data. *CoRR*, abs/1807.02547, 2018.
- Daniel E. Worrall, Stephan J. Garbin, Daniyar Turmukhambetov, and Gabriel J. Brostow. Harmonic networks: Deep translation and rotation equivariance. *arXiv preprint*, 2016. URL <https://arxiv.org/abs/1612.04642>.
- Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. Supermasks in superposition. *arXiv preprint*, 2020. URL <https://arxiv.org/abs/2006.14769>.
- Mitchell Wortsman, Maxwell C Horton, Carlos Guestrin, Ali Farhadi, and Mohammad Rastegari. Learning neural network subspaces. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 11217–11227. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/wortsman21a.html>.
- Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1912–1920, 2015. doi: 10.1109/CVPR.2015.7298801.
- Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017.
- Ruijia Xu, Guanbin Li, Jihan Yang, and Liang Lin. Unsupervised domain adaptation: An adaptive feature norm approach. *CoRR*, 2018. URL <http://arxiv.org/abs/1811.07456>.

- Ruijia Xu, Guanbin Li, Jihan Yang, and Liang Lin. Larger norm more transferable: An adaptive feature norm approach for unsupervised domain adaptation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 1426–1435, 2019.
- Fanny Yang, Zuowen Wang, and Christina Heinze-Deml. Invariance-inducing regularization using worst-case transformations suffices to boost accuracy and spatial robustness. In *ICML 2019 Workshop on Identifying and Understanding Deep Learning Phenomena*, 2019. URL <https://openreview.net/forum?id=Bl6oy39aE>.
- Pew-Thian Yap, Xudong Jiang, and Alex Chichung Kot. Two-dimensional polar harmonic transforms for invariant image representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(7):1259–1270, 2010. doi: 10.1109/TPAMI.2009.119.
- Chengxi Ye, Xiong Zhou, Tristan McKinney, Yanfeng Liu, Qinggang Zhou, and Fedor Zhdanov. Exploiting invariance in training deep neural networks. *arXiv preprint*, 2021. URL <https://arxiv.org/abs/2103.16634>.
- Shaokai Ye, Kailu Wu, Mu Zhou, Yunfei Yang, Sia Huat Tan, Kaidi Xu, Jiebo Song, Chenglong Bao, and Kaisheng Ma. Light-weight calibrator: a separable component for unsupervised domain adaptation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 13736–13745, 2020.
- Richard Zhang. Making convolutional networks shift-invariant again. *arXiv preprint*, 2019. URL <http://arxiv.org/abs/1904.11486>.
- Yingxue Zhang and Michael Rabbat. A Graph-CNN for 3D point cloud classification. *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018. URL <https://arxiv.org/abs/1812.01711>.

APPENDIX

A THEORETICAL RESULTS

For a continuous transformation $T(\alpha)$, we provide theoretical results that help to understand the structure of the β -space using continuity arguments of the optimal solution space.

A.1 CONTINUITY OF α -TO- β SUBSPACE MAPPING

To simplify the formulation of the theorems and proofs, we suppose that the set of admissible parameters $\theta \in \mathbb{T}$ is a bounded subspace of $\mathbb{R}^L$ and all optimal parameter vectors θ_α^* are in the interior of $\mathbb{T}$. To support theoretical analysis of the continuity of configuration space β , given continuous transformation parameters α , we first introduce the necessary definitions and assumptions.

Parameterized curves. We define parameterized curves $c : I \rightarrow \mathbb{R}^n$ in n -dimensional Euclidean space that are at least one time continuously differentiable, where I is a non-empty interval of real numbers. $t \in I$ is parameterizing the curve where each $c(t)$ is a point on c . In the following, we suppose that $c(t)$ is a natural parameterization of curve c , i.e., the arc length on the curve between $c(a)$ and $c(b)$ is $b - a$:

$$b - a = \int_a^b \|c'(x)\|_2 dx.$$

As the shortest curve between two points is a line, we also find that $b - a \geq \|c(b) - c(a)\|_2$. For example, let us define a curve $\alpha(s) = (1 - \frac{s}{d})\alpha^0 + \frac{s}{d}\alpha^1$ where $d = \|\alpha^1 - \alpha^0\|_2$. Then we have

$$\int_a^b \|c'(s)\|_2 ds = \int_a^b \frac{1}{d} \|\alpha^1 - \alpha^0\|_2 ds = b - a.$$

Assumptions. At first, we are interested in the relation between α and corresponding optimal parameter vectors θ_α^* , i.e., parameter vectors that minimize $E(\theta, \alpha)$. In order to simplify the forthcoming discussions, we suppose that the set of admissible parameters $\theta \in \mathbb{T}$ is a bounded subspace of $\mathbb{R}^L$ and all optimal parameter vectors θ_α^* are in the interior of $\mathbb{T}$. We assume that the loss function $E(\theta, \alpha)$ is differentiable w.r.t. to θ and α , and that it satisfies the Lipschitz condition

$$|E(\theta, \alpha_2) - E(\theta, \alpha_1)| \leq K_\alpha \|\alpha_2 - \alpha_1\|_2 \quad (3)$$

for some finite constant K_α , and for all $\alpha_1, \alpha_2 \in \mathbb{A}$ and $\theta \in \mathbb{T}$.

We are given data transformation parameters $\alpha \in \mathbb{A}$ and a network parameter vector θ_α^* . A point $\theta(0) = \theta_\alpha^*$ is a local minimum of the loss function if there is no curve segment $\theta(t)$ with $t \in [0, \hat{t}]$ for some $\hat{t} > 0$ where $E(\theta(t), \alpha) \leq E(\theta(0), \alpha)$ and $E(\theta(\hat{t}), \alpha) < E(\theta(0), \alpha)$. All curves with $t \in [0, \hat{t}]$ for some $\hat{t}$ where $E(\theta(t), \alpha) = E(\theta(0), \alpha)$ define a maximal connected subset of locally optimal parameter values. In principle, the loss landscape for a given α may contain many disconnected subsets with local minima, i.e., there is no path with a constant loss function between the locally minimal subsets.

The analysis of the loss-landscape of (over-parameterized networks) is still an active area of research, see for example (He & Tao, 2020; Kawaguchi et al., 2019; Liu, 2022). It turns out that in the case of over-parameterized networks, typical optimization methods like SGD do not get stuck in local minima when they exist, see for example (Allen-Zhu et al., 2019; Kawaguchi & Sun, 2021). Therefore, it is reasonable to assume that all local minima found by the optimizer are also global, i.e., for any given $\alpha \in \mathbb{A}$ the values of the loss functions $E(\theta_\alpha^*, \alpha)$ for all local minima θ_α^* are equal.

Relation between data and network transformation. Loosely speaking, the following theorem shows that for any continuous curve that connects two transformation parameters in $\mathbb{A}$ there exists a corresponding continuous curve in the network parameter space $\mathbb{T}$. These two curves completely map onto each other where the network parameters are optimal for the corresponding data transformations. In particular, the curve in the network parameter space $\mathbb{T}$ has no jumps as is continuous.

Theorem A.1. Suppose that the loss function $E(\theta, \alpha)$ satisfies (3), then the following holds: For any continuous curve $\alpha(s) \in \mathbb{A}$ with $0 \leq s \leq \hat{s}$ in the parameter space of data transformations there exists a corresponding curve $\theta(t) \in \mathbb{T}$ with $0 \leq t \leq \hat{t}$ in the parameter space of network parameters and a relation $(s, t) \in R$ such that

- the domain and range of R are the intervals $s \in [0, \hat{s}]$ and $t \in [0, \hat{t}]$, respectively, and
- the relation R is monotone, i.e., if $(s_1, t_1), (s_2, t_2) \in R$ then $(s_1 \geq s_2) \Rightarrow (t_1 \geq t_2)$, and
- for every $(s, t) \in R$ the network parameter vector $\theta(t)$ minimizes the loss function $E(\theta, \alpha)$ for the data transformation parameter $\alpha(s)$.

Proof. At first let us define a connected region $\mathbb{B}(\theta_\alpha^*, \delta, \alpha)$ of δ -minimal loss functions values for given transformation parameters α and corresponding locally optimal network parameters θ_α^* , where $\theta_\alpha^* \in \mathbb{B}(\theta_\alpha^*, \delta, \alpha)$ and

$$\mathbb{B}(\theta_\alpha^*, \delta, \alpha) = \{\theta \mid E(\theta, \alpha) - E(\theta_\alpha^*, \alpha) \leq \delta\}. \quad (4)$$

In other words, within the connected region $\mathbb{B}(\theta_\alpha^*, \delta, \alpha)$ that contains θ_α^* , the loss function is at most δ larger than the optimal loss $E(\theta_\alpha^*, \alpha)$. Note that $\{\theta \mid E(\theta, \alpha) - E(\theta_\alpha^*, \alpha) \leq \delta\}$ may be the union of many connected regions, but $\mathbb{B}(\theta_\alpha^*, \delta, \alpha)$ is the unique connected region that contains θ_α^* .

In order to prove the theorem, we show first that a small change in the data transformation from $\alpha(s)$ to $\alpha(s + \epsilon)$ leads to a new optimal network parameter vector $\theta_{\alpha(s+\epsilon)}^*$ that is within $\mathbb{B}(\theta_\alpha^*, \delta, \alpha)$, and δ decreases with the amount of change in α . More precisely, we show the following statement: Given transformation parameters $\alpha(s)$, corresponding optimal network parameters $\theta_{\alpha(s)}^*$, and neighboring transformation parameters $\alpha(s + \epsilon)$ with a distance ϵ on the curve. Then the new optimal network parameter vector $\theta_{\alpha(s+\epsilon)}^*$ corresponding to $\alpha(s + \epsilon)$ is within the δ -minimal region of $\theta_{\alpha(s)}^*$, namely

$$\theta_{\alpha(s+\epsilon)}^* \in \mathbb{B}(\theta_{\alpha(s)}^*, \delta, \alpha(s)) \quad (5)$$

if $\delta > 2\epsilon K_\alpha$. Therefore, for an infinitesimally distance ϵ on the α -curve, the new optimal network parameter vector $\theta_{\alpha(s+\epsilon)}^*$ is within the δ -minimal region around $\theta_{\alpha(s)}^*$. Furthermore, there exists a curve segment between $\theta_{\alpha(s)}^*$ and $\theta_{\alpha(s+\epsilon)}^*$ where every point θ on this curve satisfies $E(\theta, \alpha(s)) - E(\theta_{\alpha(s)}^*, \alpha(s)) \leq \delta$ according to (4), i.e., its loss for $\alpha(s)$ is at most δ higher than the loss at the beginning of the curve segment. Such a curve segment always exists as $\mathbb{B}$ is a connected region and the curve segment can completely be within $\mathbb{B}$. The change in loss δ for $\alpha(s)$ on the curve segment decreases with ϵ and is infinitesimally small.

We now prove the above statement. From (3) we find

$$|E(\theta, \alpha(s + \epsilon)) - E(\theta, \alpha(s))| \leq K_\alpha \|\alpha(s + \epsilon) - \alpha(s)\|_2 \leq \epsilon K_\alpha. \quad (6)$$

First, we show that the minimum for $\alpha(s + \epsilon)$ is within the region $\mathbb{B}(\theta_{\alpha(s)}^*, \delta, \alpha(s))$. At the border of the region we find $E(\theta, \alpha(s + \epsilon)) \geq E(\theta_{\alpha(s)}^*, \alpha(s)) + \delta$ due to (4). Combining this with (6) yields $E(\theta, \alpha(s + \epsilon)) \geq E(\theta_{\alpha(s)}^*, \alpha(s)) + \delta - \epsilon K_\alpha$. In the interior of the region we find as the best bound $E(\theta, \alpha(s + \epsilon)) \leq E(\theta_{\alpha(s)}^*, \alpha(s)) + \epsilon K_\alpha$ using (6). If the loss for $\alpha(s + \epsilon)$ is larger at the border of the region than in its interior, we know that a locally minimal loss is within the region, i.e., (5) holds. Therefore, we require $E(\theta_{\alpha(s)}^*, \alpha(s)) + \epsilon K_\alpha < E(\theta_{\alpha(s)}^*, \alpha(s)) + \delta - \epsilon K_\alpha$ and therefore, $\delta > 2\epsilon K_\alpha$.

Using the above statement, see (5), we start from some curve $\alpha(s)$, $0 \leq s \leq \hat{s}$ and construct a corresponding optimal curve in the network parameter space $\theta(t)$ for $0 \leq t \leq \hat{t}$. We begin with some $\alpha(s)$ and an optimal network $\theta_{\alpha(s)}^* \in \arg \min E(\theta, \alpha(s))$. We know that the optimal parameter vector $\theta_{\alpha(s+\epsilon)}^*$ for infinitesimally close transformation parameters $\alpha(s + \epsilon)$ on the curve $\alpha(s)$ is within the δ -minimal region around $\theta_{\alpha(s)}^*$. Therefore, to a small segment from $\alpha(s)$ to $\alpha(s + \epsilon)$ we assign a finite segment from $\theta_{\alpha(s)}^*$ to $\theta_{\alpha(s+\epsilon)}^*$ completely within the δ -minimal region around $\theta_{\alpha(s)}^*$. Every point on this curve segment corresponds to a network whose loss is either infinitesimally close to the optimal values $E(\theta_{\alpha(s)}^*, \alpha(s))$ or $E(\theta_{\alpha(s+\epsilon)}^*, \alpha(s + \epsilon))$. In other words, the curve segment starts from optimal network parameters $\theta_{\alpha(s)}^*$, ends at optimal network parameters $\theta_{\alpha(s+\epsilon)}^*$, and in between traverses the region with loss values that are infinitesimally close to either of these optimal loss values. This process is repeated, starting from $\alpha(s + \epsilon)$ and $\theta_{\alpha(s+\epsilon)}^*$. As a result, the two curves $\alpha(s)$ and $\theta(t)$ are connected by a relation $(s, t) \in R$ such that the domains are the intervals of the curve parameters $[0, \hat{s}]$ and $[0, \hat{t}]$. If $(s, t) \in R$ then $\theta(t)$ is optimal for $\alpha(s)$. No points on the curves are missing, i.e., without a relation to the other curve. Moreover, the relation R is monotone: If $(s_1, t_1), (s_2, t_2) \in R$ then $(s_1 \geq s_2) \Rightarrow (t_1 \geq t_2)$. $\square$

Note that the assumption that local minima are also global minima is crucial. For example, suppose that for a given α there are two local minima that are separated by a single barrier. Suppose further that by changing α , just the height of the barrier reduces until it vanishes completely. At this value of α , a small change, i.e., a short distance on the curve

$\alpha(s)$, leads to a large change in the optimal θ . In other words, given a curve $\alpha(s)$ there may be no corresponding continuous curve $\theta(t)$ that satisfies the properties of the above theorem.

We are also interested in the relation between α and corresponding optimal vectors β_α^* that define optimal locations on the linear subspace of admissible network parameters as defined by (1). To simplify the discussion, we suppose that $\beta \in \mathbb{B}$ are in a bounded subspace of $\mathbb{R}^D$, and all basis vectors θ_j that define $f(\beta)$ in (1) have bounded elements. Under these assumptions, we can derive a corollary from Theorem A.1.

Corollary A.2. *Suppose that the loss function $E(\theta, \alpha)$ satisfies (3), and for any $\alpha \in \mathbb{A}$ all local minima of $E(f(\beta), \alpha)$ w.r.t. β are global. Then the following holds: For any continuous curve $\alpha(s) \in \mathbb{A}$ with $0 \leq s \leq \hat{s}$ in the parameter space of data transformations there exists a corresponding curve $\beta(t) \in \mathbb{B}$ with $0 \leq t \leq \hat{t}$ on the linear network parameter subspace according to (1) and a relation $(s, t) \in R$ such that*

- the domain and range of R are the intervals $s \in [0, \hat{s}]$ and $t \in [0, \hat{t}]$, respectively, and
- the relation R is monotone, i.e., if $(s_1, t_1), (s_2, t_2) \in R$ then $(s_1 \geq s_2) \Rightarrow (t_1 \geq t_2)$, and
- for every $(s, t) \in R$ the network parameter vector $\beta(t)$ minimizes the loss function $E(f(\beta), \alpha)$ for the data transformation parameter $\alpha(s)$.

Proof. Sketch: The relation between the parameterization β of the linear subspace spanned by θ_i and the resulting network parameters $\theta = f(\beta)$ is given by (1). As $\beta \in \mathbb{B}$ is bounded and the basis vectors θ_j are finite, $f(\theta)$ is differentiable and Lipschitz constrained. Therefore, the proof as provided for Theorem A.1 holds as well by just replacing $\theta = f(\beta)$. Therefore, the results of Theorem A.1 hold for β as well if for any $\alpha \in \mathbb{A}$ all local minima of $E(f(\beta), \alpha)$ w.r.t. β are global. $\square$

A.2 SMALL PARAMETER CHANGES

This section shows that small changes to transform parameters α result in small changes of optimal configuration β_α^* for suitable loss functions $E(f(\beta), \alpha)$. For the forthcoming analysis, we suppose that $E(f(\beta), \alpha)$ is at least twice differentiable w.r.t. α and β .

Theorem A.3. *Suppose that $\beta_0^* \in \mathbb{B}$ locally minimizes $E(f(\beta), \alpha)$ for $\alpha_0 \in \mathbb{A}$. Moreover, the Hessian $\nabla_{(\beta, \alpha)}^2 E(f(\beta), \alpha)$ of the loss function at $\alpha = \alpha_0$ and $\beta = \beta_0^*$ exists, and its submatrix $\nabla_\beta^2 E(f(\beta), \alpha)$ is non-singular.*

Then, if $\|\alpha_1 - \alpha_0\| \leq \epsilon$ for some small ϵ , then there exists a locally optimal β_1^ for α_1 such that*

$$\|\beta_1^* - \beta_0^*\| \leq \|(\nabla_\beta^2 E(f(\beta), \alpha))^{-1}\| \cdot \|\nabla_\alpha (\nabla_\beta E(f(\beta), \alpha))\| \cdot \epsilon.$$

Proof. For a network trained to a minimum in $E(f(\beta), \alpha)$ for a given α , its first derivative over weights equals to 0, i.e., $\nabla_\beta E(f(\beta), \alpha) = 0$ for an optimal vector β_α^* . We assume that this derivative exists and is abbreviated by a function $F(\beta, \alpha) = \nabla_\beta E(f(\beta), \alpha)$.

Let $F(\beta, \alpha)$ at $\beta = \beta_\alpha^*$ be a differentiable function of β and α . We apply the first-order Taylor expansion of F at a point (β_0^*, α_0) , i.e., β_0^* is optimal for α_0 :

$$F(\beta, \alpha) = F(\beta_0^*, \alpha_0) + \nabla_{\beta^*} F \Big|_{(\beta_0^*, \alpha_0)} \delta\beta^* + \nabla_\alpha F \Big|_{(\beta_0^*, \alpha_0)} \delta\alpha, \quad (7)$$

where $\delta\beta^* = \beta_1^* - \beta_0^*$ and $\delta\alpha = \alpha_1 - \alpha_0$. We have $F(\beta^*, \alpha) = 0$ and $F(\beta_0^*, \alpha_0) = 0$ due to the optimality of the loss function.

We abbreviate the evaluated partial derivatives $\nabla_{\beta^*} F \Big|_{(\beta_0^*, \alpha_0)} := P$ and $\nabla_\alpha F \Big|_{(\beta_0^*, \alpha_0)} := Q$. Since β^* and α are vectors, we find that $P \in \mathbb{R}^{D \times D}$ and $Q \in \mathbb{R}^{S \times D}$ are matrices. Thus,

$$P \cdot \delta\beta^* + Q \cdot \delta\alpha = 0 \quad (8)$$

and therefore,

$$\delta\beta^* = -P^{-1}Q\delta\alpha.$$

Using basic results from linear algebra we find

$$\|\delta\beta^*\| \leq \|P^{-1}\| \cdot \|Q\| \cdot \|\delta\alpha\|$$

and therefore

$$\|\beta_1^* - \beta_0^*\| \leq \|P^{-1}\| \cdot \|Q\| \cdot \|\alpha_1 - \alpha_0\| \leq \|P^{-1}\| \cdot \|Q\| \cdot \epsilon.$$

From the last equation it follows that small changes of transform parameters $\delta\alpha$ result in small changes $\delta\beta^*$ of the optimal solution β_0^* if P is invertible. Note that $P = \nabla_\beta^2 E(f(\beta), \alpha)$ and $Q = \nabla_\alpha(\nabla_\beta E(f(\beta), \alpha))$ for $\alpha = \alpha_0$ and $\beta = \beta_0^*$. $\square$

Note that the condition of the theorem is crucial, *i.e.*, the Hessian of the loss function with respect to the parameters β of the linear subspace at the optimal solution is invertible. This excludes cases with saddle points, where there is no optimal point vector in the neighborhood after a small change in α . Moreover, we can only make statements about local minima of the loss function due to the use of the Taylor expansion.

B IMPLEMENTATION DETAILS

We trained over 1’000 models on a workstation featuring two NVIDIA GeForce RTX 2080 Ti GPUs to evaluate the performance of SCNs presented in this work. Training a model takes up to several hours and depends on the SCN dimensionality and model complexity. WandB⁵ was used to log hyper-parameters and output metrics from runs.

B.1 DATASETS AND ARCHITECTURES

Configuration network. Throughout all experiments we used the configuration network featuring one fully-connected layer comprising 64 neurons. Depending on the input (whether 2 values for 2D rotation and translation, 6 values for 3D rotation, and 1 value for all other considered transformations), the configuration network contains $64 \cdot (\text{input_size} + 1) + 65 \cdot D$ trainable parameters. Note that D is the size the the configuration network’s output. The architecture includes the bias term. For example, for 2D rotation with 3 outputs, the configuration network has 387 parameters.

We test the performance of SCNs on five dataset-architecture pairs described below. For MLPs and ShallowCNNs we vary architectures’ width and depth to understand the impact of network capacity on efficiency of SCNs for different D . To scale up along the width dimension, we double the number of neurons in each hidden layer. When increasing depth, we increase the number of layers of the same width. To improve training efficiency for deeper networks (deeper than 3 layers), we use BatchNorm layers (Ioffe & Szegedy, 2015) when scaling up MLPs and ShallowCNNs along the depth dimension. The number of parameters for the network architectures specified below (excluding BatchNorm parameters) is only for a single inference network $\mathcal{G}$. D base models of this size are learned when training a SCN.

MLPs on FMNIST. FMNIST (Xiao et al., 2017) is the simplest dataset considered in this work. The dataset includes 60’000 images for training and 10’000 images for testing. The dataset is available under the MIT License.⁶ We use MLPs of varying width w and depth l to evaluate the impact of the dense network capacity on SCNs. The number of parameters of the MLP inference network for 10 output classes scales as follows:

$$(32^2 + 1) \cdot w + (l - 1) \cdot (w^2 + w) + 10 \cdot (w + 1).$$

ShallowCNNs on SVHN. SVHN (Netzer et al., 2011) digit classification dataset contains 73’257 digits for training, 26’032 digits for testing, and 531’131 additionally less difficult digits for assisting training. No additional images are used. The dataset is available for non-commercial use.⁷ Shallow convolutions (ShallowCNNs) were introduced by Neyshabur (2020). We scale the architecture along the width w and depth d dimensions. The number of parameters scales as follows:

$$(9 \times 9 \times 3 + 1) \cdot w + (l - 1) \cdot (13 \times 13 \times w + 1) \cdot w + 10 \cdot (w + 1).$$

LeNet5 on ModelNet10. ModelNet10 (Wu et al., 2015) is a subset of ModelNet40 comprising a clean collection of 4,899 pre-aligned shapes of 3D CAD models for objects of 10 categories. We use this dataset to evaluate SCN performance on images of 3D rotated objects. We first rotate an object in the 3D space, subsample a point cloud from the rotated object, which is then projected to a fixed plane. The projection is then used as input to the inference network. Rotation parameters α are input to the trained hypernetwork to obtain the parameters in the β -space to construct an optimal inference network. We use LeNet-5 (Lecun et al., 1998) as inference network architecture with 138’562 parameters.

⁵<https://wandb.ai>

⁶<https://github.com/zalandoresearch/fashion-mnist>

⁷<http://ufldl.stanford.edu/housenumbers/>

ResNet18 on CIFAR10. This work adopts the ResNet18 implementation by He et al. (2015) with around 11 million trainable parameters. We use ResNet18 on CIFAR10 (Krizhevsky et al., 2009), one of the most widely used datasets in machine learning research. The dataset comprises 60’000 color images from 10 classes and is publicly available.⁸

M5 on Google Speech Commands. We explore the performance of SCNs in the audio signal processing domain by adopting M5 (Dai et al., 2016) convolutional architecture to classify keywords in the Google Speech Commands dataset (Warden, 2018)). M5 networks are trained on the time domain waveform inputs. The dataset consists of over 105,000 WAV audio files of various speakers saying 35 different words and is available under the Creative Commons BY 4.0 license. It is part of the Pytorch common datasets.⁹

B.2 TRAINING HYPER-PARAMETERS

Table 2 summarizes the set of hyper-parameters used to train different networks throughout this work.

Table 2: **Training hyper-parameters** for all architecture-dataset pairs.

Hyper-param.	MLP-FMNIST	ShallowCNN-SVHN	ResNet18-CIFAR10	LeNet5-ModelNet10	M5-SpeechCmds
Optimizer	Adam	Adam	Adam	Adam	Adam
LR	0.001	0.001	0.001	0.006	0.01
Weight decay			0.0001		
LR schedule	CosineLR	CosineLR	CosineAnnealing WarmRestarts, $T_0 = 25$, $T_{mult} = 25$	CosineAnnealing $T_{max} = 6’000$, $\eta_{min} = 5 \cdot 10^{-6}$	CosineAnnealing $T_{max} = 100$, $\eta_{min} = 0$
Batch size	64	256	512	256	256
Epochs	500	500	1’000	6’000	100
Data augment.	Normali- zation	Normali- zation	Normalization, HorizontalFlip	None	Resample to 16 KHz

B.3 TRANSFORMATIONS

This paper evaluates SCNs on the following computer vision and audio signal transformations: 2D rotation, scaling, translation, 3D rotation-and-projection, brightness, contrast, saturation, sharpness, pitch shift and speed change described below. Figure 10 illustrates examples of transformations applied to a sample input, showcasing various non-obvious effects that result in a decrease in input quality. Consequently, this decrease in quality adversely affects the accuracy of a trained classifier.

2D rotation. The rotation transformation is parameterized by a single angle ϕ in the range $0-2\pi$. We use $\alpha = (\cos(\phi), \sin(\phi))$ as input to the configuration network when learning SCNs for 2D rotations. The transformation preserves distances and angles, yet may lead to information loss due to cropped image corners and rounding effects. It can be inverted with little loss of image quality, as can be observed in Figure 10.

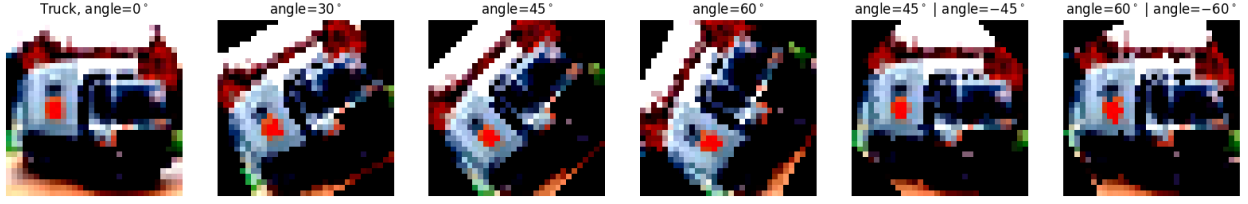
Scaling. The transformation is parameterized by the scaling factor in the range $0.2-2.0$, which is input to the hypernetwork to learn the configuration β -space for this transformation. Scaling transformation leads to a considerable loss of image quality. When inverted, the image appears highly pixelated or cropped.

Translation. We consider image shifts within the bounds $(-8, -8)$ and $(8, 8)$ pixels. A shift is represented by two parameters $\alpha = (\alpha_x, \alpha_y)$ reflecting the shift along the x and y axes. Note that an image gets cropped when translation is undone. In the FMNIST dataset the feature objects are positioned at the center of the image, which mitigates the negative impact of translations compared to other datasets like SVHN and CIFAR10.

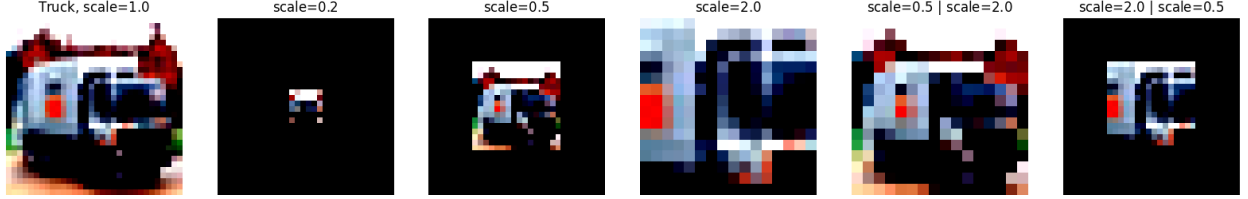
3D rotation. The 3D rotation transformation is parameterized by the three Euler angles that vary in the range $(-\pi, \pi)$. We use $\alpha = (\cos(\phi_1), \sin(\phi_1), \cos(\phi_2), \sin(\phi_2), \cos(\phi_3), \sin(\phi_3))$ as the input to the hypernetwork for learning SCNs

⁸<https://www.cs.toronto.edu/~kriz/cifar.html>

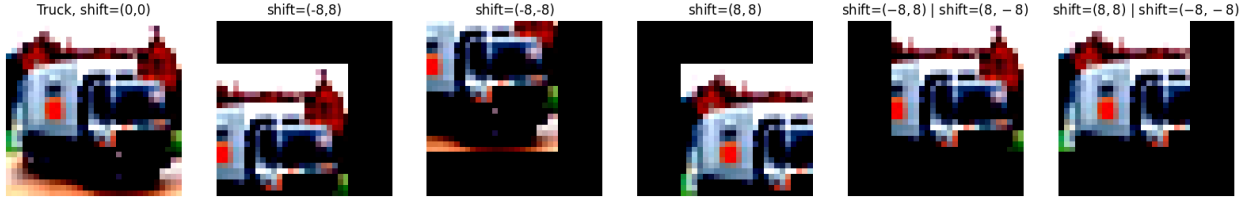
⁹<https://pytorch.org/audio/stable/datasets.html>



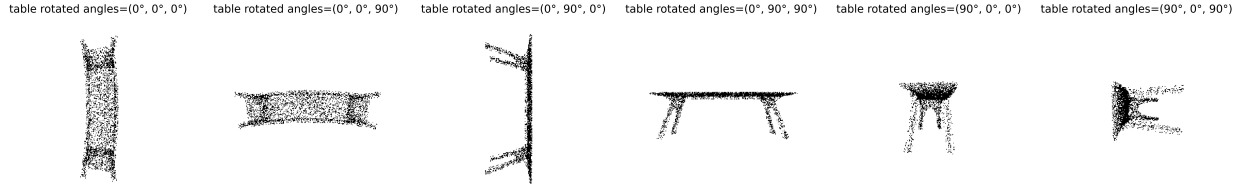
(a) **2D rotation transformation parameterized by an angle ϕ in the range $(0-2\pi)$.** The transformation preserves angles and distances and can be undone with little loss of image quality (the edges of the input image may get cropped, rounding effects may occur).



(b) **Scaling transformation parameterized by a scaling factor in the range $(0.2-2.0)$.** Preserves only angles, not fully invertible, reduces input quality, large portions of the input image may get cropped.



(c) **Translation transform with a shift in $(-8,-8)-(8,8)$.** Fully invertible only for the part of the input image inside the middle square $(8,8)$ to $(24,24)$.



(d) **3D rotation transform.** We rotate an object in 3D along XY, YZ, and XZ planes using 3 angles (ϕ_1, ϕ_2, ϕ_3) , $\phi_i \in (-\pi, \pi)$ and sample a point cloud of 4'096 points. Rotations in XZ (e.g., angles= $(0, \frac{\pi}{2}, 0)$) and YZ (e.g., angles= $(\frac{\pi}{2}, 0, 0)$) planes can block some pixels (e.g., the table surface, which is not visible in the picture).

Figure 10: **Geometric transformations** used in this work applied to a sample input. Notice how the images get impacted when inverse transformation is applied, showing a loss of input quality due to rounding, re-scaling and cropping.

on 3D rotations. Note that a different order of the same combination of these three angles may produce a different transformation output. We apply a fixed order (ϕ_1, ϕ_2, ϕ_3) in all 3D rotation experiments. After rotation the 3D point cloud is projected on a 2D plane. When applying 3D rotations, it is possible to lose pixels in cases where the rotation axis is parallel to the projection plane. An example is shown in Figure 10.

Color transformations. We explore SCN performance on four common color transformations: brightness, contrast, saturation and sharpness. The *brightness* parameter governs the amount of brightness jitter applied to an image and is determined by a continuously varying brightness factor. The *contrast* parameter influences the distinction between light and dark colors in the image. *Saturation* determines the intensity of colors present in an image. Lastly, *sharpness*

controls the level of detail clarity in an image. We vary the continuously changing α parameter between 0.2 and 2.0 for all considered color transformations.

Audio signal transformations. We use SCNs with pitch shift and speed adjustment transformations. Pitch shift modifies the pitch of an audio frame by a specified number of steps, with the parameter adjusted within the range of -10 to +10. Similarly, speed adjustment alters the playback speed by applying an adjustment factor, with speed changes applied within the range of 0.1 to 1.0.

C CONFIGURATION SUBSPACES AND SCN EFFICIENCY

C.1 SCN PERFORMANCE

SCN performance on geometric transformations. Figure 11 complements Figure 2 in the main paper and presents the performance of SCNs for 2D rotation on ShallowCNN–SVHN and ResNet18–CIFAR10. We observe high efficiency of SCNs compared to the baselines even for low D . A close inspection of models for a fixed degree ($\phi = 0^\circ$) shows their increasingly higher specialization to the respective transformation parameter setting.

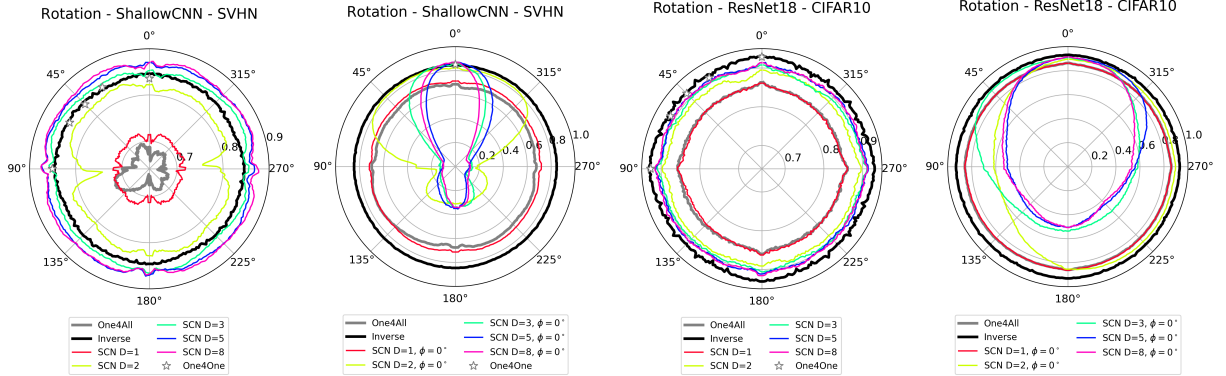


Figure 11: **SCN test set accuracy on 2D rotations.** From left to right: A pair of plots for ShallowCNN–SVHN and ResNet18–CIFAR10. The models in each pair show SCN’s performance for changing input $\alpha = (\cos(\phi), \sin(\phi))$ and for a fixed α with $\phi = 0^\circ$.

SCN performance on color transformations. Color transformations are simple. SCNs achieve high performance already for very low $D = 2$ or $D = 3$ (see Figure 12). There is little performance difference between the baselines One4All, One4One and Inverse despite the small inference model size (1-layer MLP with 32 hidden units for FMNIST and 2-layer ShallowCNN with 32 channels for SVHN).

SCN performance on audio signal transformations. Figure 13 shows the performance of SCNs on two audio signal transformations: pitch shift and speed change. For both transformations a low D is sufficient for SCN to match or outperform the baselines. Note that M5 takes a raw waveform in the time domain as input rather than a spectrogram.

C.2 CONFIGURATION β -SPACE VISUALIZATION

Different dataset-architecture pairs exhibit a similar structure in the β -space. Figure 14 and Figure 15 present the learned configuration parameters β as a function of the transformation parameters α for 2D rotation and scaling, respectively, complementing the findings in Figure 4 of the main paper. It is worth noting the slight variations in the shape of the learned curves, which are specific to the architecture-dataset pairs used to train SCNs. Based on the consistent β -space across different dataset-architecture pairs, we infer that the configuration space primarily relies on the transformation and the characteristics of its parameter vector α .

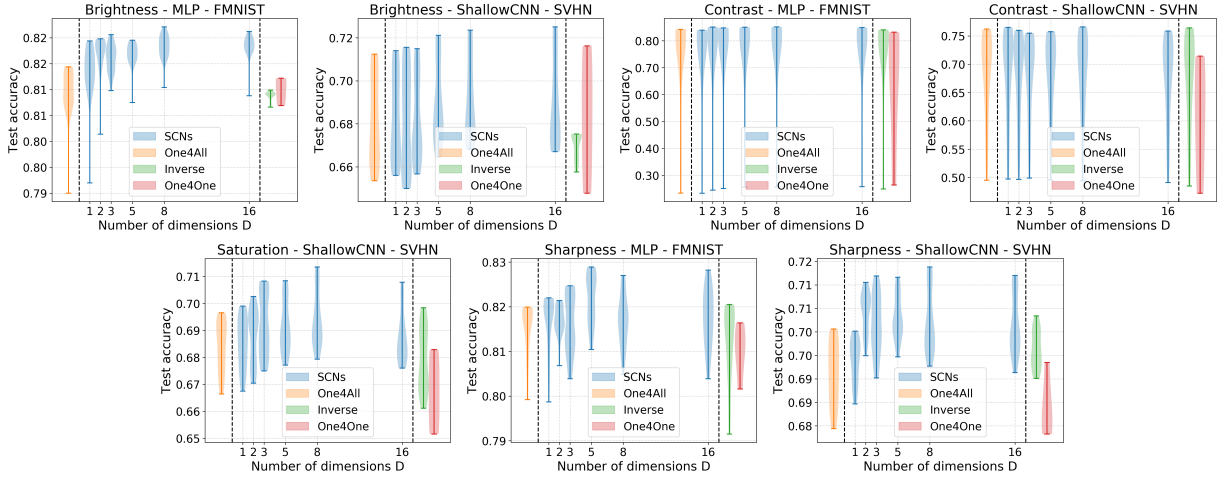


Figure 12: **Summary of SCN performance on color transformations:** brightness, contrast, saturation and sharpness. We present the results for MLP-FMNIST and ShallowCNN-SVHN architecture-dataset pairs. All transformations are simple. SCNs match the baselines for very low D . Note that saturation has no effect on black-white images. Therefore, for MLP-FMNIST the difference in model performance is the same up to the choice of a random seed.

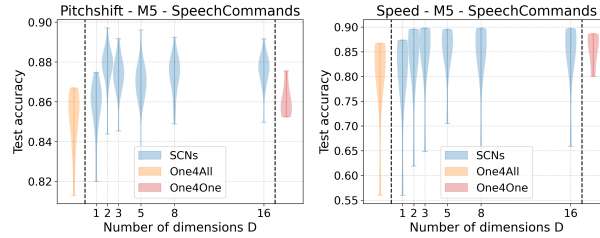


Figure 13: **Summary of SCN performance on audio signal transformations:** pitch shift and speed using M5 as inference architecture. SCNs match the performance of baselines already for small D .

C.3 SCN AT A BLOCK LEVEL

We investigate the performance of SCNs when applied to just one block of ResNet18 on CIFAR10 for 2D rotation transformations in Figure 16. Applying SCN solely on the 4th block yields results comparable to SCN’s application across all blocks. However, applying SCN exclusively to the 1st block results in inferior performance.

C.4 COMPARISON TO DOMAIN ADAPTATION METHODS

Since robustness to input transformations can be framed as a domain shift problem, we compare SCNs with two domain adaptation baselines on the 2D-rotation task on FMNIST using LeNet-5 network architecture in Table 3. The first baseline is the Stepwise Adaptive Feature Norm (SAFN) (Xu et al., 2019), which defines a distance measure between the source and the target domains in the feature space, and minimizes this distance when training the network to reduce

Model	FMNIST Rotated[%] degree=10	FMNIST Rotated[%] degree=90	FMNIST Rotated[%] degree=0...360
SAFN	42.41 $\pm$ 6.93	9.61 $\pm$ 0.54	10.11 $\pm$ 0.16
Data Calibrator	77.16 $\pm$ 0.75	9.28 $\pm$ 0.67	19.77 $\pm$ 0.39
SCN(D=3)[ours]	86.15 $\pm$ 0.21	86.75 $\pm$ 0.35	73.75 $\pm$ 2.40
SCN(D=5)[ours]	86.51 $\pm$ 0.56	86.92 $\pm$ 0.77	85.71 $\pm$ 3.35

Table 3: Comparison of SCNs to SAFN and Data Calibrator for models adapted from FMNIST to the target domain. With $D=5$ SCN surpasses SAFN and Data Calibrator in multiple target domains

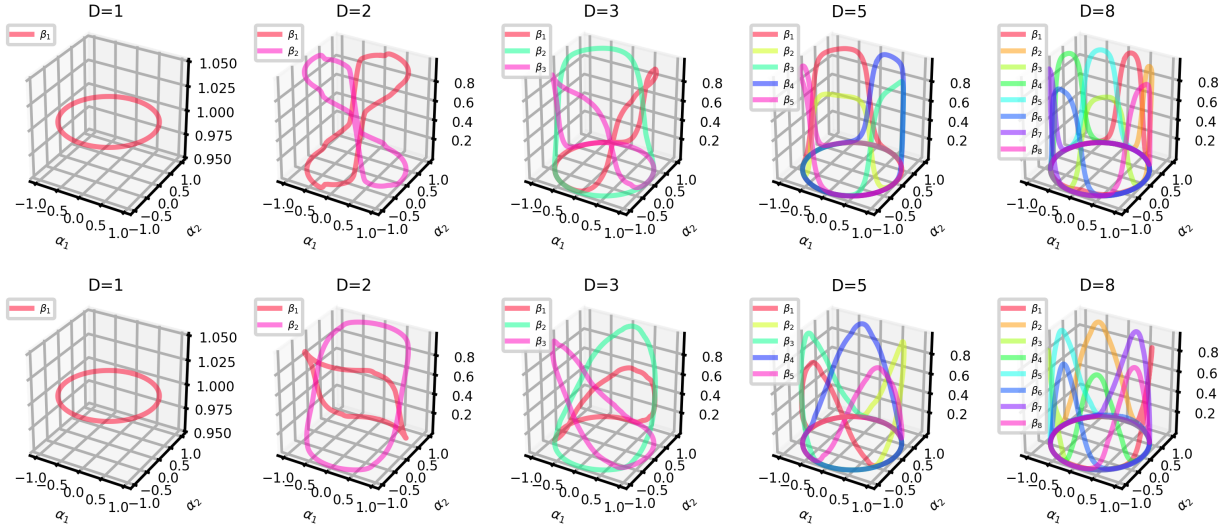


Figure 14: **Configuration β -space of SCNs trained for 2D rotation** on further datasets and inference network architectures, complementing Figure 4. **Top:** 1-layer MLP with 32 hidden units on FMNIST. **Bottom:** 2-layer ShallowCNNs with 32 filters in the hidden layers on SVHN. Transform parameters are $\alpha = (\alpha_1, \alpha_2) = (\cos(\phi), \sin(\phi))$, with ϕ being a rotation angle.

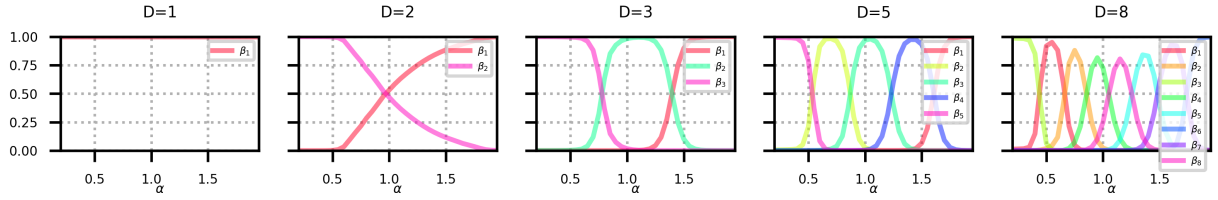


Figure 15: **Configuration β -space of SCNs trained for scaling using MLP on FMNIST**. Scaling factor α is between 0.2–2.0. The β -space looks similarly to the one shown in the main paper for a different dataset-architecture pair.

the domain shift. Furthermore, we compare SCNs to the Data Calibrator (Ye et al., 2020), which fixes the source classifier and recovers discrimination power in the target domain, while preserving the source domain’s performance.

D TRANSLATION TRANSFORMATION

SCNs on architectures NOT invariant to translation. Figure 17 shows SCN performance on translation transformation for 1-layer MLP with 32 hidden units as inference network trained on the FMNIST dataset. SCN’s test accuracy increases with higher D matching the accuracy of the Inverse baseline. The visualization allows identifying high accuracy areas of each base model. With higher D , the area of a dedicated model for a specific parameter setting decreases, whereas its test accuracy increases.

SCNs on translation-invariant architectures. Using translation-invariant architecture as inference network part of SCN trained for translation results in a degenerated β -space with only one base model. This special case is exemplified in Figure 18. On the FMNIST dataset side, we scale the input images down by 50% and apply padding of 8 to ensure that shifting the image within $(-8, 8)$ along horizontal and vertical axes leads to a pure translation of the object in the image without information loss. As translation-invariant network architecture, we use a 2-layer CNN built only of convolutional and max pooling layers with kernel size of 4 and 16 channels. For any D , SCNs learn a single model with only one $\beta_i=1$ and other $\beta_j, j \neq i$ being zero. The checkered structure of the test accuracy plot reflects the size of the filters. A detailed explanation of its origin and its relation to the Nyquist–Shannon sampling theorem is given in (Zhang, 2019).

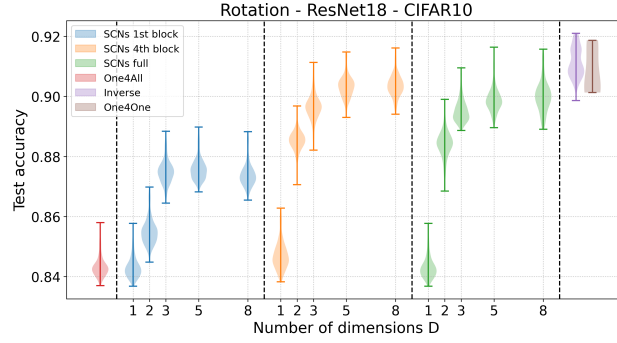


Figure 16: textbfSCN at a block level. Performance results for the 2D rotation transformation applied to ResNet18 on CIFAR10. Applying SCN solely on the 4th block yields results comparable to SCN application across all blocks. Applying SCN exclusively to the 1st block results in inferior performance.

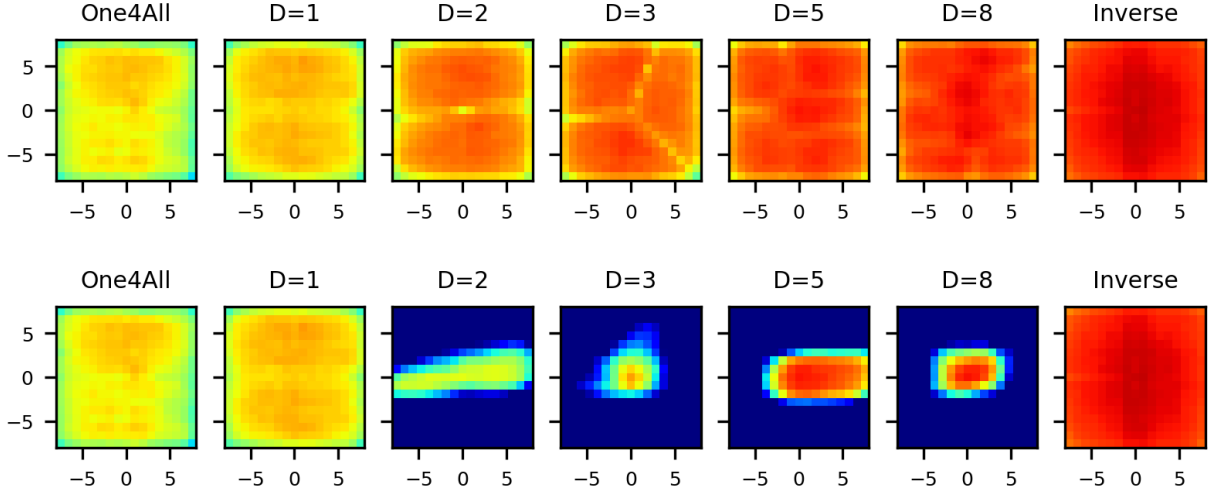


Figure 17: **SCN performance for translation trained with MLP inference network on FMNIST.** In this example, applying translation to an input image leads to information loss, since the part of the image shifted outside the image boundary gets cut. We use 1-layer MLPs with 32 hidden units and a bias term. This architecture is not translation-invariant. In all plots the color map is "rainbow" ranging uniformly between 0.5 or below (dark blue) to 0.9 (dark red). X and Y axes are horizontal and vertical shift parameters (α_x, α_y) applied to the input. **Top:** Test accuracy of SCNs for $D = 1..8$ for every (α_x, α_y) combination, compared to One4All and Inverse baselines. **Bottom:** Test accuracy of SCNs for the dedicated fixed (0,0) shift evaluated on shifted inputs. The area of high accuracy decreases with higher D , leading to higher degree of model specialization, higher accuracy of the dedicated model for each setting, and a better overall performance of SCNs.

E 3D ROTATION TRANSFORMATION

Figure 19 shows all views of the β -space of SCN for 3D rotation as a function of input parameters $\alpha = (\cos(\phi_1), \sin(\phi_1), \cos(\phi_2), \sin(\phi_2), \cos(\phi_3), \sin(\phi_3))$, where $\phi_i, i = 1..3$ is a rotation angle in the YZ, XZ and XY planes, respectively. Figure 19 shows the whole β -space for 3D rotation presented as a function of all pairwise combinations of ϕ_i . In Figure 19 middle and bottom, β s show a stable trend along the ϕ_3 -axis, indicating that the 3D rotation in the XY plane keeps all object pixels (and is basically the same as 2D rotation in this case). In Figure 19 (top), β -space has cosine-like trend along both ϕ_1 and ϕ_2 axes, reflecting the 3D rotations in YZ and XZ planes. These transformations lead to information loss as some parts of an object rotate out of the view and get blocked. In all plots β -surfaces are not flat or degenerated. By observing the similarities and changing trends in the learned β -space for 3D rotation, it can be inferred that the shape of this configuration space primarily relies on the transformation itself and its

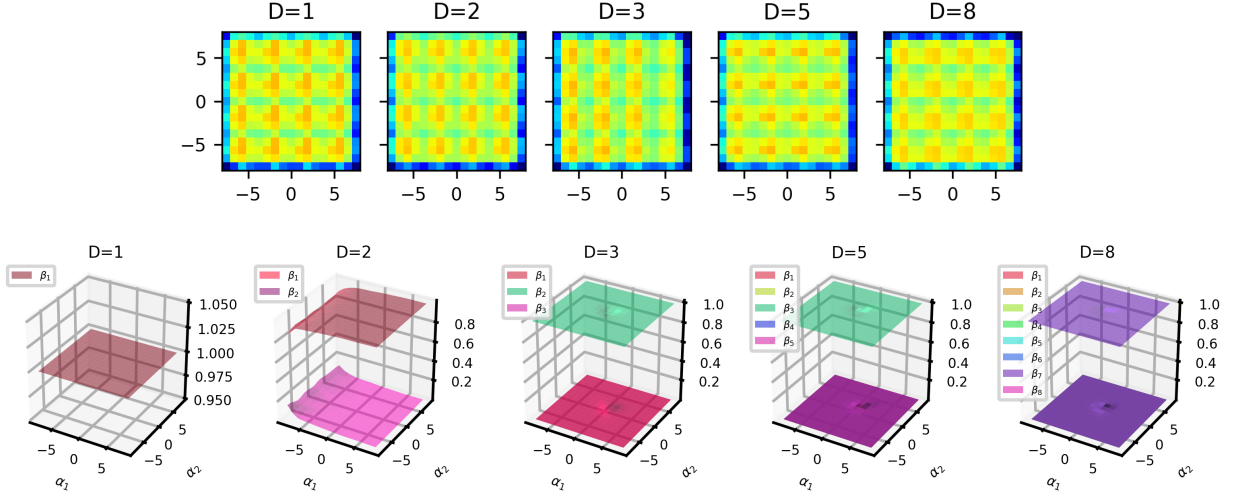


Figure 18: **SCN performance for translation trained on translation-invariant CNN architecture on FMNIST.** SCNs for all D learn a degenerated β -space with only one active model (only one $\beta_i=1$) for all inputs. **Top:** Independently trained SCNs for different D yield very similar accuracy of 0.85. The checkered structure of the plots reflects the size of the filters, which is 4x4. **Bottom:** Configuration β -space showing that only one β_i equals 1.0 for all input parameters α .

associated parameters, namely (ϕ_1, ϕ_2, ϕ_3) . We provide a link¹⁰ to an interactive website visualizing the β -space of sample SCNs, including those trained for 3D rotation.

F SEARCH ALGORITHM IN THE α -SPACE

This section provides details on the performance of the search algorithm which estimates α from a stream of input data. As mentioned in the main paper, we can leverage the fact that the correct input parameters α should produce a confident low-entropy classification result (Wortsman et al., 2020; Hendrycks & Gimpel, 2016). Therefore, our search algorithm estimates α from a batch of input data by minimizing the entropy of the model output on this batch by exploring the output of optimal models in the learned low-dimensional subspace. We use the basin hopping¹¹ method to find the solution (with default parameters, iter=100, T=0.1, method=BFGS).

The following code snippet runs the search in the α -space to estimate the best rotation angle α from a batch of data X by minimizing the function $f()$. The angle transformation function converts an input angle in degrees to the corresponding $(\cos, \sin)$ pair.

```
from scipy import optimize

# function to be minimized by the basin hopping algorithm
def f(z, *args):
    alpha = transform_angle(((1+z)*180)%360-180)
    X = args[0]
    logits = model(Tensor(X), hyper_x=Tensor(alpha))
    b = (F.softmax(logits, dim=1)) * (-1 * F.log_softmax(logits, dim=1)) # entropy
    return b.sum().numpy()

# given a batch of images find the rotation angle alpha using basin hopping algorithm
def findalpha(X):
    kwargs = {"method": "BFGS", "args": X}
    res = optimize.basinhopping(f, 0.0, minimizer_kwargs=kwargs, niter=100, T=0.1)
    alpha = ((1+res.x[0])*180)%360-180
    return alpha

# test search algorithm performance on a test set
```

¹⁰<https://subspace-configurable-networks.pages.dev/>

¹¹<http://tinyurl.com/yp9ve2dd>

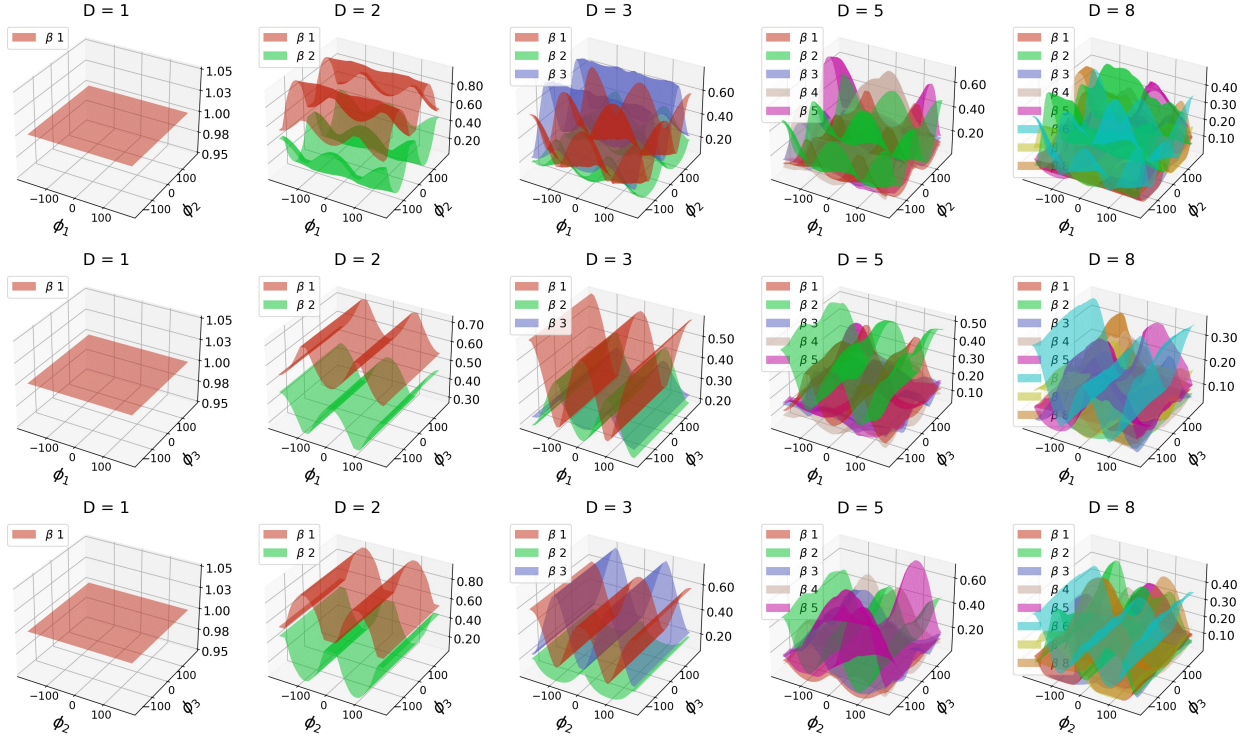


Figure 19: $\alpha - \beta$ space of SCNs trained for 3D rotation on ModelNet10 with LeNet5 inference architecture for $D = 1..8$. Transformation parameters α result from applying $\cos(\cdot)$ and $\sin(\cdot)$ functions to the vector of rotation angles (ϕ_1, ϕ_2, ϕ_3) , with each ϕ_i in the range $(-\pi, \pi)$. **Top:** Subspace of SCNs when changing (ϕ_1, ϕ_2) , and fixing $\phi_3 = -\pi$. **Middle:** Subspace of SCNs when changing (ϕ_1, ϕ_3) , and fixing $\phi_2 = -\pi$. **Bottom:** Subspace of SCNs when changing (ϕ_2, ϕ_3) , and fixing $\phi_1 = -\pi$.

```

result = 0.0
for (X, y) in test_loader:
    angle = random.uniform(-180, 180)
    X = TF.rotate(X, angle)

    alpha = findalpha(X)

    # compute model prediction with the estimated alpha
    logits = model(X, hyper_x=transform_angle(alpha))
    # y is the true label --> calculate accuracy
    correct = (logits.argmax(1) == y).type(torch.float).sum().item() / batch_size
    result += correct

result /= len(test_loader.dataset) / batch_size
print(f"Test accuracy: {(100*result):>0.1f}%")

```

To improve the accuracy of the search, SCN training is enhanced with an additional regularizer to minimize the model output entropy value for the correct α and maximise it for a randomly sampled α . The train function is sketched in the listing below.

```

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=args.lr)
cos = nn.CosineSimilarity(dim=0, eps=1e-6)

def train(dataloader, model, loss_fn, optimizer):
    for (X, y) in dataloader:
        X, y = X.to(device), y.to(device)

```

```

angle = random.uniform(0, 360)
X = TF.rotate(X, angle)

# make prediction and compute the loss
pred = model(X, hyper_x=transform_angle(angle).to(device))
loss = loss_fn(pred, y)

# regularize (cosine similarity squared) in the beta space
beta1 = model.hyper_stack(transform_angle(angle).to(device))
angle2 = random.uniform(0, 360)
beta2 = model.hyper_stack(transform_angle(angle2).to(device))
loss += pow(cos(beta1, beta2), 2)

# minimize entropy for the correct degree
b1 = (F.softmax(pred, dim=1)) * (-1 * F.log_softmax(pred, dim=1))
loss += 0.01*b1.sum()

# maximize entropy for a wrong / random degree
logits = model(X, hyper_x=transform_angle(angle2).to(device))
b2 = (F.softmax(logits, dim=1)) * (-1 * F.log_softmax(logits, dim=1))
loss -= 0.01*b2.sum()

optimizer.zero_grad()
loss.backward()
optimizer.step()

```

The interested reader can check the source code for further details.¹²

Note that the basin hopping algorithms is computationally expensive. For the 2D rotation transformation on FMNIST dataset, the method may run several hundreds of model inferences to find a good solution. Optimizing the running time of the method is beyond the scope of this paper, because in practice α -search can be avoided, *e.g.*, by using an additional sensor modality as input or by discretizing the search space to a manageable number of models. The expensive α -search aims to show that the problem of estimating α and building I-SCNs is solvable in principle.

G SCNS ON LOW-RESOURCE DEVICES VISION TASK

In Section 3.6, we showcase and measure the SCN’s performance on fruit classification using RGB sensor. In this section, we show SCNs’ performance on classifying traffic signs from 2D-rotated images on IoT devices. For all tasks, additional sensor data is used to derive the input α , elevating the need to perform α -search on the device. The data gathered in our experiments is online.¹³

We evaluate the models’ test accuracy on the held-out datasets. On IoT devices, we compare SCN to the original One4All baseline, as well as to the wider and deeper One4All variants. To evaluate the time-efficiency of SCN, we measure the latency of three phases: executing the configuration network to obtain the vector β (“Hypernet Inference”), computing θ from base models θ_i (“Configuration”), and executing the inference network $\mathcal{G}$ (“Inference”). All reported times present averages over 100 measurements. The running time of the One4All model is solely determined by its inference time. We measure the required storage capacity by separately quantifying the flash and RAM usage of the SCN models, original One4All models, and modified One4All models. Our embedded experiments utilize two MCUs: Tensilica Xtensa 32-bit LX7 dual-core for traffic sign classification and nRF52840 for fruit classification. One copy of the initialized model weights is stored in flash and loaded into RAM upon program start. The D base models of SCNs are stored exclusively in flash, thereby conserving valuable RAM resources.

G.1 TRAFFIC SIGN CLASSIFICATION

The German Traffic Sign Benchmark contains 39,209 images of 43 traffic signs captured on German roads (Houben et al., 2013). During training, we rotate each traffic sign image at an arbitrary angle, supplied as α to the SCN’s configuration network. We evaluate SCN’s performance on 12,630 self-gathered traffic sign images, each fixed at a randomly chosen rotation angle. These images were collected using the onboard camera of the ESP32S3-EYE

¹²<https://github.com/osaukh/subspace-configurable-networks/>

¹³<https://github.com/osaukh/subspace-configurable-networks/tree/main/IoT>

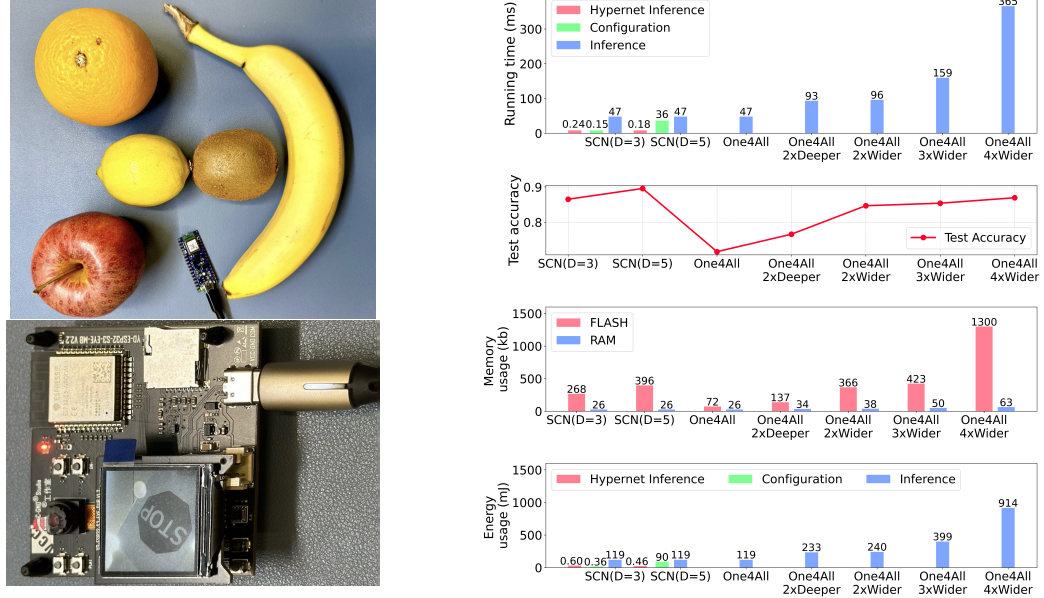


Figure 20: **SCN performance on the traffic sign classification task on ESP32-S3-EYE**, along with the visualization of both setups of the fruit classification task (see section 3.6) and traffic sign classification (left). From top to bottom the plots show: (1) Inference time in milliseconds, showcasing the efficiency of SCN, where the deeper and wider One4All variants lead to increased inference times. For SCNs, we also measure the execution latencies of the configuration network used to obtain vector β ("Hypernet Inference"), and the computation time for generating θ from base models θ_i ("Configuration"). These latencies are only incurred when the deployment environment changes. (2) Test accuracy across various architectures, highlighting SCN's highly competitive performance. (3) RAM and flash memory usage in kB, indicating the increased resource consumption as the One4All model expands. (4) Energy consumption in mJ.

development board featuring Tensilica Xtensa 32-bit LX7 dual-core processor with 8 MB Octal PSRAM and 8 MB flash (Espressif, 2023). The data measured with the on-board IMU sensor of the ESP32S3-EYE development board is used to calculate the rotation angle during testing.

Figure 20 (right) presents the performance of $\text{SCN}(D=3)$ and $\text{SCN}(D=5)$ on classifying traffic signs from 2D-rotated images. The One4All model used in this experiment shares the same architecture as SCN: It features a sequence of three fully connected layers, with 12, 8, and 5 neurons respectively, each followed by a ReLU activation function. For the modified One4All models, including the $N \times \text{Deeper}$ and $N \times \text{Wider}$ modifications, the number of layers or the number of hidden units per layer are increased. Figure 20 (right) shows that the One4All architecture, when made four times wider, matches the accuracy of $\text{SCN}(D=3)$. However, the resource consumption of $\text{SCN}(D=3)$ is notably lower: inference time and energy usage are reduced by the factor of 7.6, flash usage by the factor of 4.9, and RAM usage by the factor of 2.4. The overhead of reconfiguring the SCN model constitutes only a tiny fraction of the SCN inference time and is almost negligible. Despite sharing the same network architecture, One4All may lack sufficient capacity to store all the necessary augmentations for the desired input transformations. This could result in a significant drop in accuracy, which becomes more pronounced under tighter resource constraints.

H RELATED WORK

Networks trained on extensive datasets lack robustness to common transformations of the input, such as rotation (Gandikota et al., 2021), scaling (Ye et al., 2021), translation (Biscione & Bowers, 2022) or small deformations (Engstrom et al., 2017). For example, Gong et al. (2014) showed that CNNs achieve neither rotation nor scale invariance, and their translation invariance to identify an object at multiple locations after seeing it at one location is limited (Kauderer-Abrams, 2018; Blything et al., 2020; Biscione & Bowers, 2022). Moreover, deep networks remain susceptible to adversarial attacks with respect to these transformations, and small perturbations can cause significant changes in the network predictions (Gandikota et al., 2021). There are three major directions of research to address the problem in the model design phase: Modifying the training procedure, the network architecture, or

the data representation. Alternatively, the problem can be treated as a domain adaptation challenge and solved in the post-deployment phase. Below, we summarize the related literature.

Modifying the training scheme. The methods that modify the training scheme replace the loss function $\mathcal{L}$ with a function that considers all parameters of transformations T in a range where the solution is expected to be invariant. Common choices are minimizing the mean loss of all predictions $\{G(u(x_i), \theta) | u \in T\}$ resulting in training with data augmentation (Botev et al., 2022), or maximizing the loss among all predictions leading to adversarial training (Engstrom et al., 2017). Both training schemes do not yield an invariant solution with respect to transformations such as rotation, as discussed in (Gandikota et al., 2021). The use of regularization can also improve robustness, yet provides no guarantees (Simard et al., 1991; Yang et al., 2019). Overall, modifications of the training procedure are popular in practical applications, since they do not require characterization of the transformations applied to the input data, which are often unknown and may include a mix of complex effects.

Designing invariant network architectures. Dedicated network architectures can be designed to be invariant to structured transformations based on a symmetric group action that preserves class labels. For example, it is commonly believed that convolutional neural networks are architecturally invariant to translation due to the design characteristics of their convolution and max pooling layers (Marcus, 2018; Kauderer-Abrams, 2018). However, multiple studies argue that the translation invariance of CNNs is rather limited (Biscione & Bowers, 2022; Blything et al., 2020). Nevertheless, designing invariant architectures to a particular transformation is the subject of many recent works (Weiler & Cesa, 2019; Libera et al., 2019) due to the desirable robustness properties they offer in practice (Gandikota et al., 2021; Schneuing et al., 2022).

Rotation invariant architectures play an important role in computer vision tasks. For instance, for a successful object classification, the orientation of the coordinate system should not affect the meaning of the data. Therefore, a broad research literature is devoted to designing rotation invariant and equivariant architectures. (Cohen & Welling, 2016; Marcos et al., 2016; Veeling et al., 2018) use rotated filters to achieve layer-wise equivariance to discrete rotation angles. For continuous rotations, Worrall et al. (2016) proposed circular harmonic filters at each layer. These approaches were consolidated in (Weiler & Cesa, 2019). Jaderberg et al. (2015); Tai et al. (2019) align transformed images using different methods, e.g., using principal component analysis. Wang et al. (2022) explore approximately equivariant networks which relax symmetry-preserving constraints, since the real world rarely conforms to strict mathematical symmetry either due to noisy or incomplete data. Weiler et al. (2018) and Thomas et al. (2018) propose 3D rotation equivariant kernels for convolutions. Esteves et al. (2017) propose a polar transformer network by learning a transformation in a polar space in which rotations become translations and so CNNs become effective to achieve rotation invariance. When 3D objects are presented as point clouds, this solves problems that arise due to object discretization, but leads to a loss of information about the neighbor relationship between individual points. Zhang & Rabbat (2018) add graph connections to compensate for this information loss and use graph convolutions to process the cloud points. Qi et al. (2016; 2017) additionally include hierarchical and neighborhood information.

Canonicalization of data representation. Input canonicalization is the process of converting the data into a specific form to simplify the task to be solved by a deep model. For example, by learning to map all self-augmentations of an image to similar representations is the main idea behind contrastive learning methods such as SimCLR (Chen et al., 2020) and Supervised Contrastive Learning (Khosla et al., 2020). Canonicalization can also be achieved by learning to undo the applied transformation or learning a canonical representation of the data (Kaba et al., 2022). For example, Jaderberg et al. (2015) propose a Spatial Transformer layer to transform inputs to a canonical pose to simplify recognition in the following layers. BFT layers (Dao et al., 2019) can be used to learn linear maps that invert the applied transform. Earlier works on the topic considered manual extraction of features in the input that are robust, or invariant, to the desired transformation (Manthalkar et al., 2003; Yap et al., 2010).

Domain adaptation. Robustness to input transformations can be framed as a domain shift problem (Loghmani et al., 2020). Domain adaptation methods described in the literature follow different strategies as to how they align the source and the target. For example, Xu et al. (2018) define a distance measure between source and target data in the feature space and minimize this during training the network to reduce domain shift. Russo et al. (2017) train a generator that makes the source and target data indistinguishable for the domain discriminator. Another group of methods uses self-supervised learning to reduce domain shift (Jiaolong et al., 2019). In many real world scenarios, the data from the target domain are available only in the post-deployment phase. Therefore, domain adaptation methods often face memory and computing resource constraints making the use of backpropagation too costly. Partial model updates, especially those executed sequentially, may reduce model quality (Vucetic et al., 2022).

Linear mode connectivity, generalization and robustness. In this work we show that optimal model weights that correspond to parameterized continuous transformations of the input reside in a low-dimensional linear subspace. This finding connects this work to recent research on the properties of the loss landscape and its relationship with

generalization and optimization (Geiger et al., 2019; Nguyen et al., 2018; Fort et al., 2019; Şimşek et al., 2021; Juneja et al., 2022; Entezari et al., 2021; Jordan et al., 2022). In particular, the existence of linear paths between solutions trained from independent initializations (Entezari et al., 2021), those that share a part of their learning trajectories (Frankle et al., 2020), or trained on data splits (Ainsworth et al., 2022). Wortsman et al. (2021) learn neural network subspaces containing diverse and at the same time linear mode connected (Frankle et al., 2020; Nagarajan & Kolter, 2019) solutions that can be effectively weight-space ensembled. This work builds upon and extends these works to linear mode connectivity between optimal models trained for different input transformation parameters.

Mixture-of-Experts. Mixture-of-experts (MoE) networks split model parameters into several expert modules, designed to fit specialized sub-tasks (Jacobs et al., 1991; Shazeer et al., 2017). In NLP, MoE successfully increases model capacity by adding multiple experts without exponential growth in computation cost (Gao et al., 2022; Caccia et al., 2022). MoE utilizes a routing function to assign data to one or a subset of appropriate expert modules for further processing. The router takes weighted averaging of the outputs of the experts. In contrast, the configuration network of SCNs can also be regarded as a router to effectively generate low-dimensional coefficients and configure the inference model weights from the base models.

To the best of our knowledge, this is the first work looking into model robustness and adaptation to input transformations in the face of severe resource constraints.